

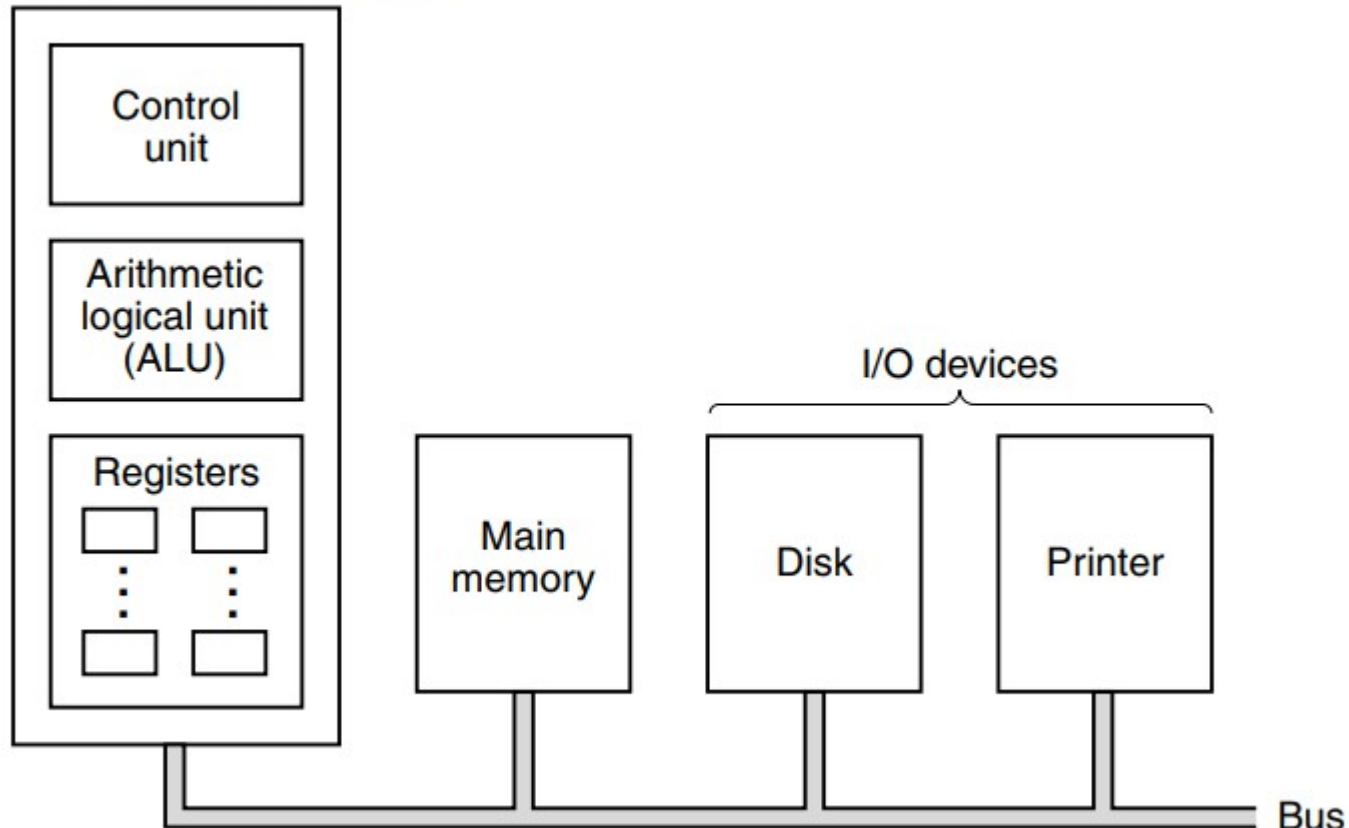
Today's Lecture Plan

- Homework8 is due today and late due date is tomorrow.
- Proj2 will be open tomorrow.
- We will learn about Virtual Memory today
 - What is virtual memory? Why we use virtual memory?
 - Virtual Address
 - Virtual Address to Physical Address Mapping/Translation
 - Page Table
 - Calculate the Size of Page Table (Polls)
 - Page Fault
 - Segmentation Fault
 - How to handle Memory Pressure?
 - How to Protect OS Data?
 - Examples of calculating Physical Address from Virtual's Address.
 - Program Walk Through
- Next Class:
 - Multilevel Page Lookup
 - TLB
 - Intel x86 Architecture and Register Model

Virtual memory

Computer organization

Central processing unit (CPU)



Questions

- What if you wanted to run a program that needs more memory than you have?
- What if you want to run 10 programs, which collectively need more memory than you have?
- What if multiple different programs want to store something at the same address?
- How do we protect one program's data from being read or written by another program, either intentionally or accidentally?
- How do we protect the operating system's data from being read or written by another program, either intentionally or accidentally?

What is Virtual Memory?

- Creates the illusion of a large, continuous memory space.
- Allows programs to exceed physical RAM.
- Backed by disk storage.

Why Use Virtual Memory?

- Memory protection and per-process isolation.
- Efficient RAM utilization.
- Supports large software systems.
- Simplifies memory allocation for developers.

What this program is doing?

```
#include <iostream>
#include <cstdlib>
#include <ctime>
#include <unistd.h>

// compile with: g++ -o test test.cpp -fPIC -fno-stack-protector -no-pie

using namespace std;
int main() {
    static int value;
    srand(time(NULL));
    value = rand();
    while(1) {
        cout << "value is " << value << " at address "
              << &value << endl;

        sleep(2);
    }
    return 0;
}
```

```
anubis

Welcome Virtual.cpp x
x86 > Virtual.cpp > main
1  #include <iostream>
2  #include <cstdlib>
3  #include <ctime>
4  #include <unistd.h>
5  // compile with: g++ -o test test.cpp -fPIC -fno-stack-protector -no-pie
6  using namespace std;
7  int main() {
8      static int value;
9      srand(seed: time(timer: NULL));
10     value = rand();
11     while(1) {
12         cout << "value is " << value << " at address "<< &value << endl;
13         sleep(seconds: 10);
14     }
15     return 0;
16 }
17

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
anubis@anubis-ide : ~/x86
● [0] % g++ -o test Virtual.cpp -fPIC -fno-stack-protector -no-pie
anubis@anubis-ide : ~/x86
○ [0] % ./test
value is 291928414 at address 0x404080
value is 291928414 at address 0x404080

anubis@anubis-ide : ~/x86
○ [0] %
anubis@anubis-ide : ~/x86
○ [0] % ./test
value is 1834822177 at address 0x404080
value is 1834822177 at address 0x404080
```

Can you find anything interesting here?

**At Address 0x404080, two different values are stored.
We can only store one value at one memory cell.**

Address space

Address space is the set of memory addresses, and their corresponding memory cells, that are available to a program.

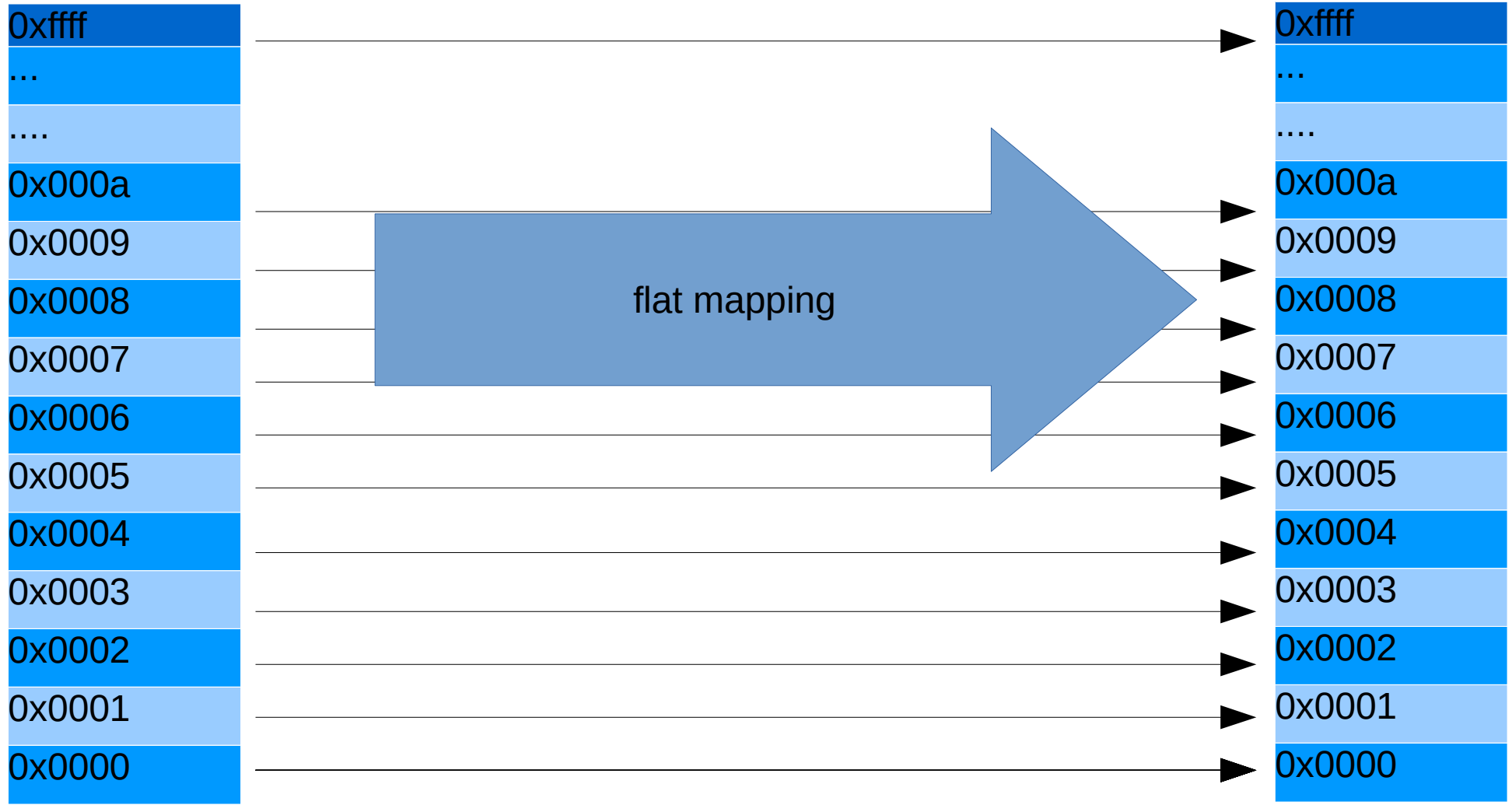
On 64-bit Computer (such as Intel), the address space is expressed as a 64-bit pointer value, so it has 2^{64} memory cell, starting from zero.

On 32-bit Machine, the address space is expressed as a 32-bit pointer value, so it has 2^{32} memory cell, starting from zero.

On E20, the address space is expressed as a 16-bit pointer value, so it has 2^{16} elements, starting from zero.

Flat address space mapping

lw \$1, foo(\$0)

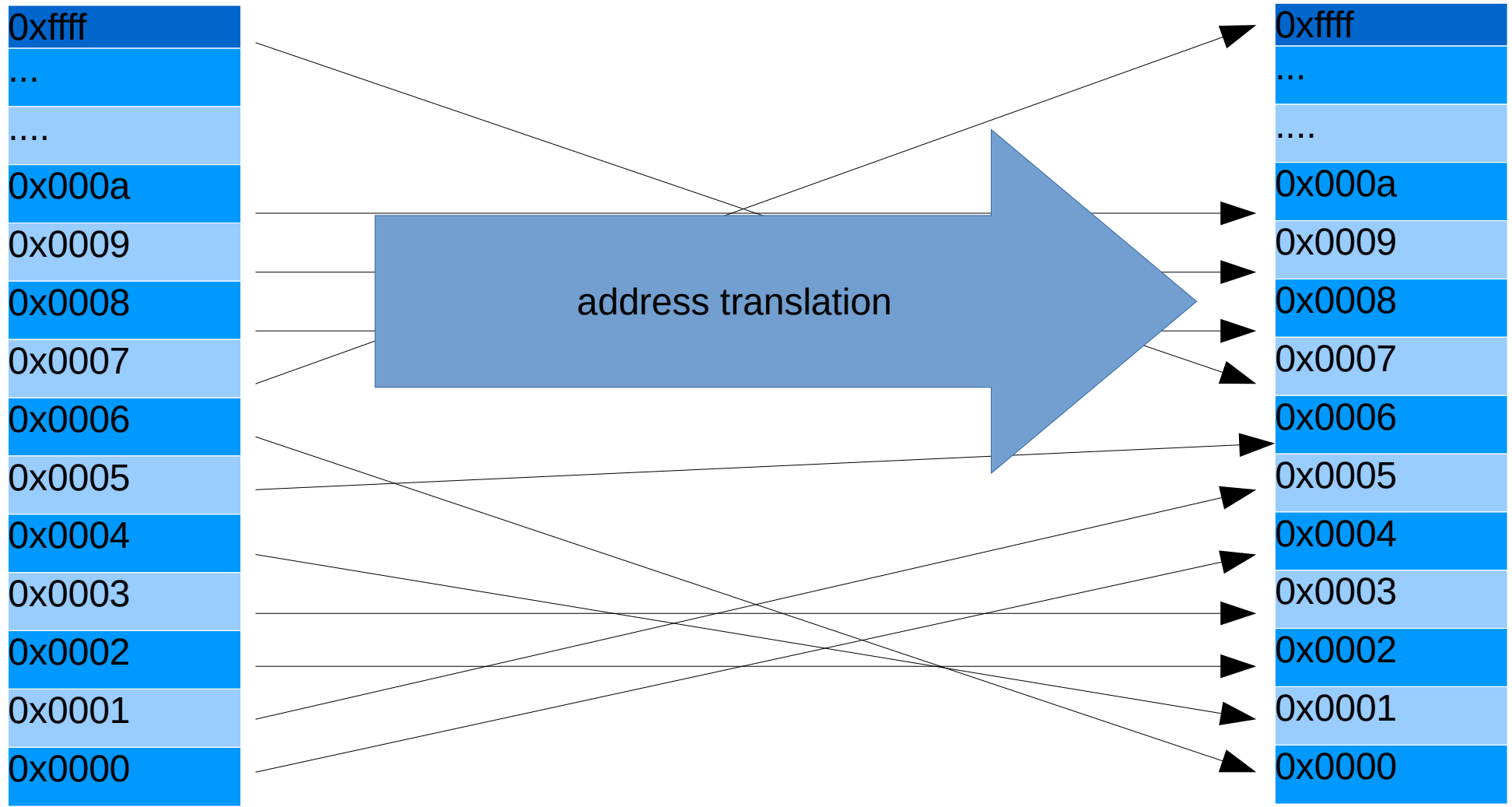


application address space

physical memory

Virtual address space mapping

lw \$1, foo(\$0)



application address space

physical memory

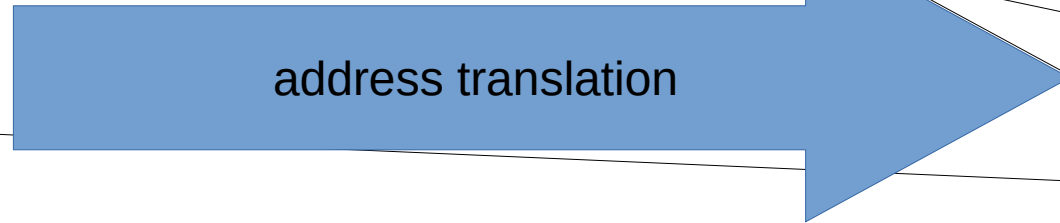
Virtual address space mapping

0xffff
...
....
0x0004
0x0003
0x0002
0x0001
0x0000

application 1
address space

0xffff
...
....
0x0004
0x0003
0x0002
0x0001
0x0000

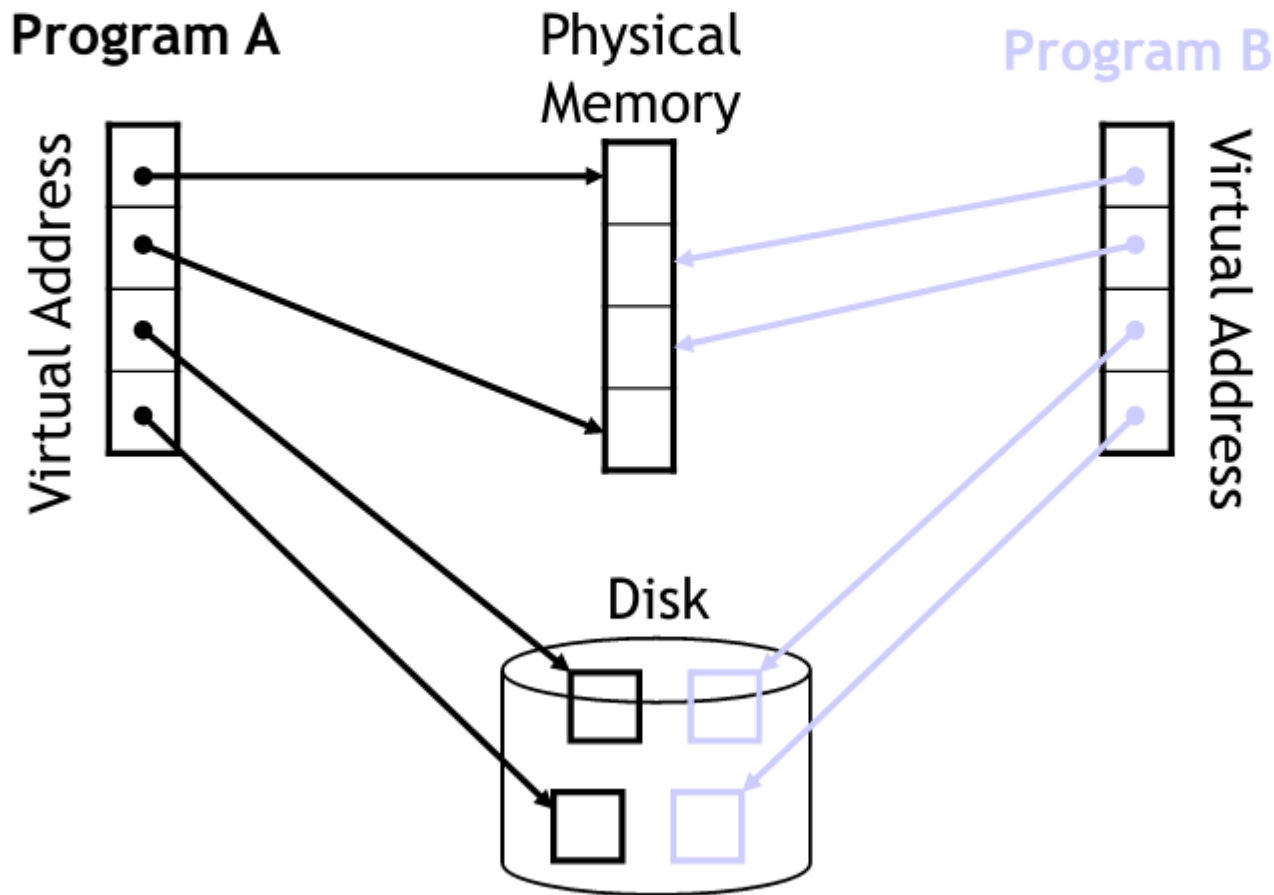
application 2
address space



0xffff
...
....
0x000a
0x0009
0x0008
0x0007
0x0006
0x0005
0x0004
0x0003
0x0002
0x0001
0x0000

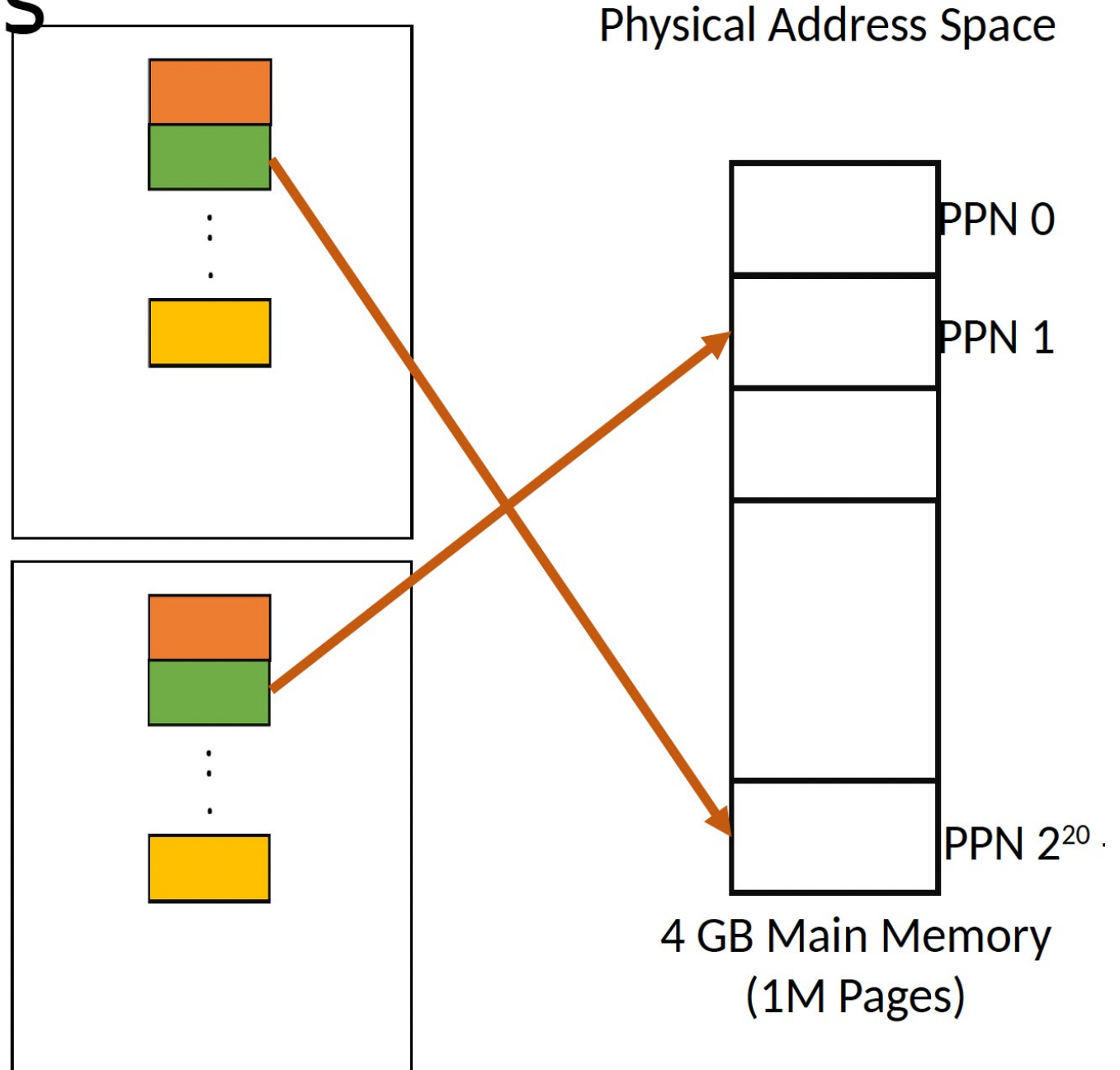
physical memory

- Because different processes will have different mappings from virtual to physical addresses, two programs can freely use the same virtual address.
- By allocating distinct regions of physical memory to A and B, they are prevented from reading/writing each others data.



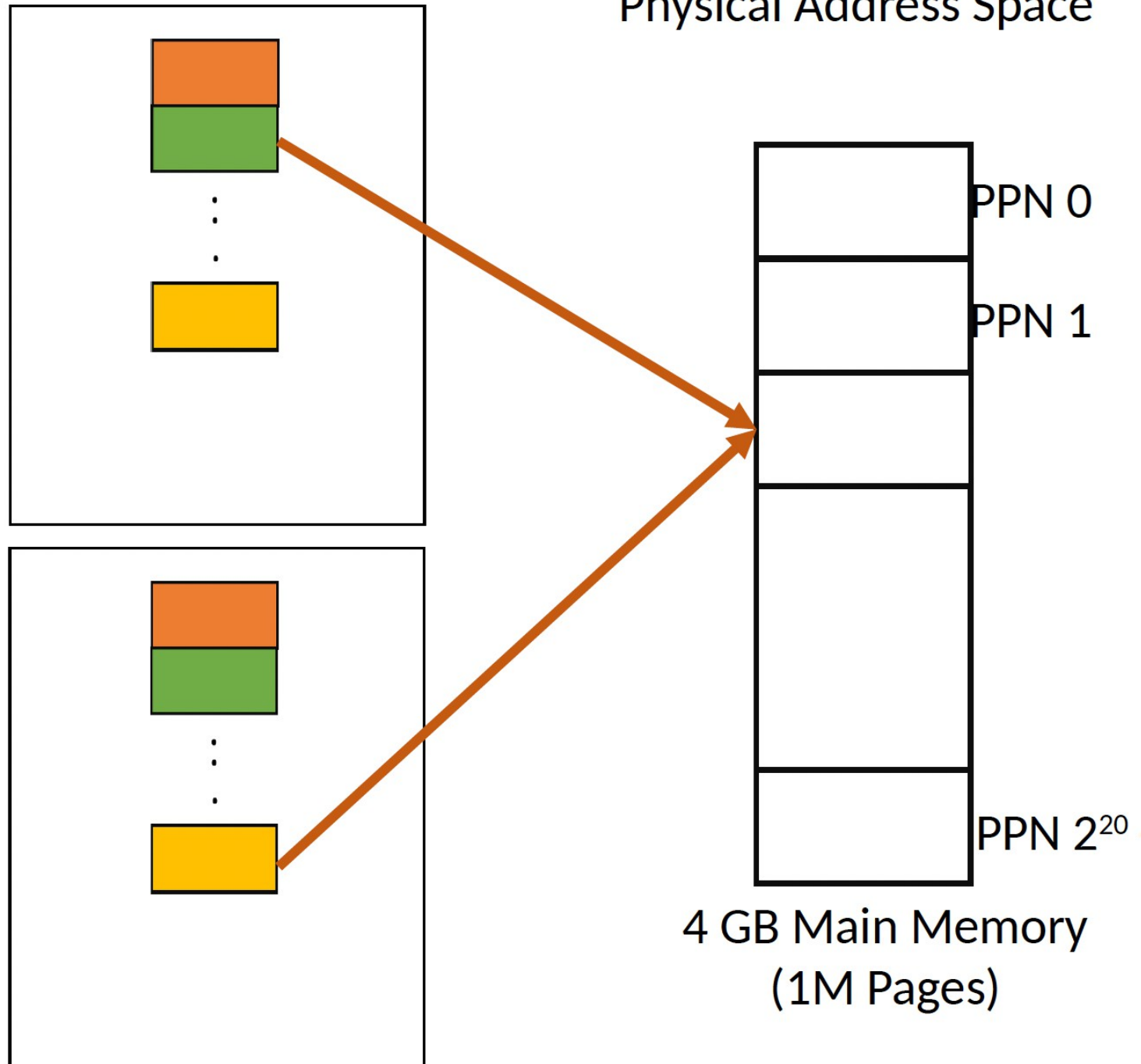
Homonyms

- The same VA (in different processes) can map to two different physical addresses

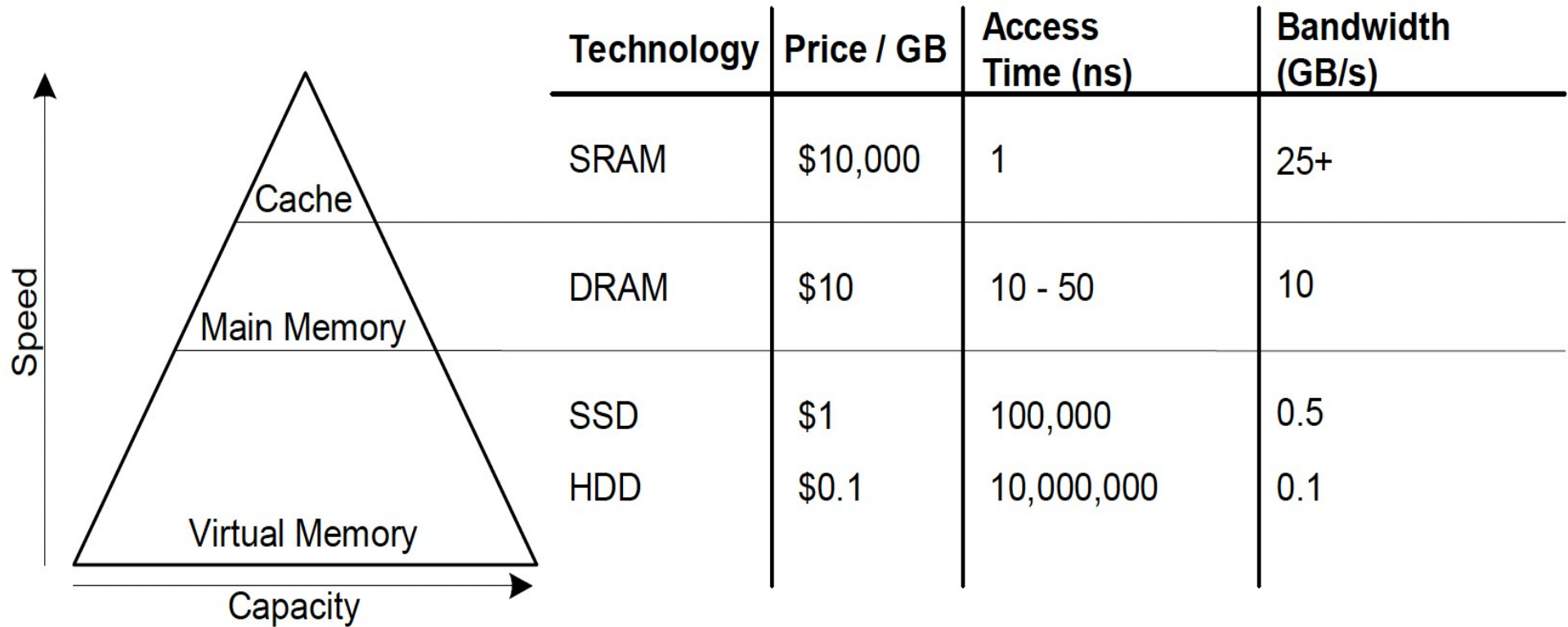


Synonyms

- Different VAs can map to the same physical address
- Shared code and data



Memory Hierarchy



SRAM, DRAM – volatile
SSD, HDD – non-volatile

Virtual Address Space Mapping

lw \$1, foo(\$0)

Virtual address space
can be larger than physical

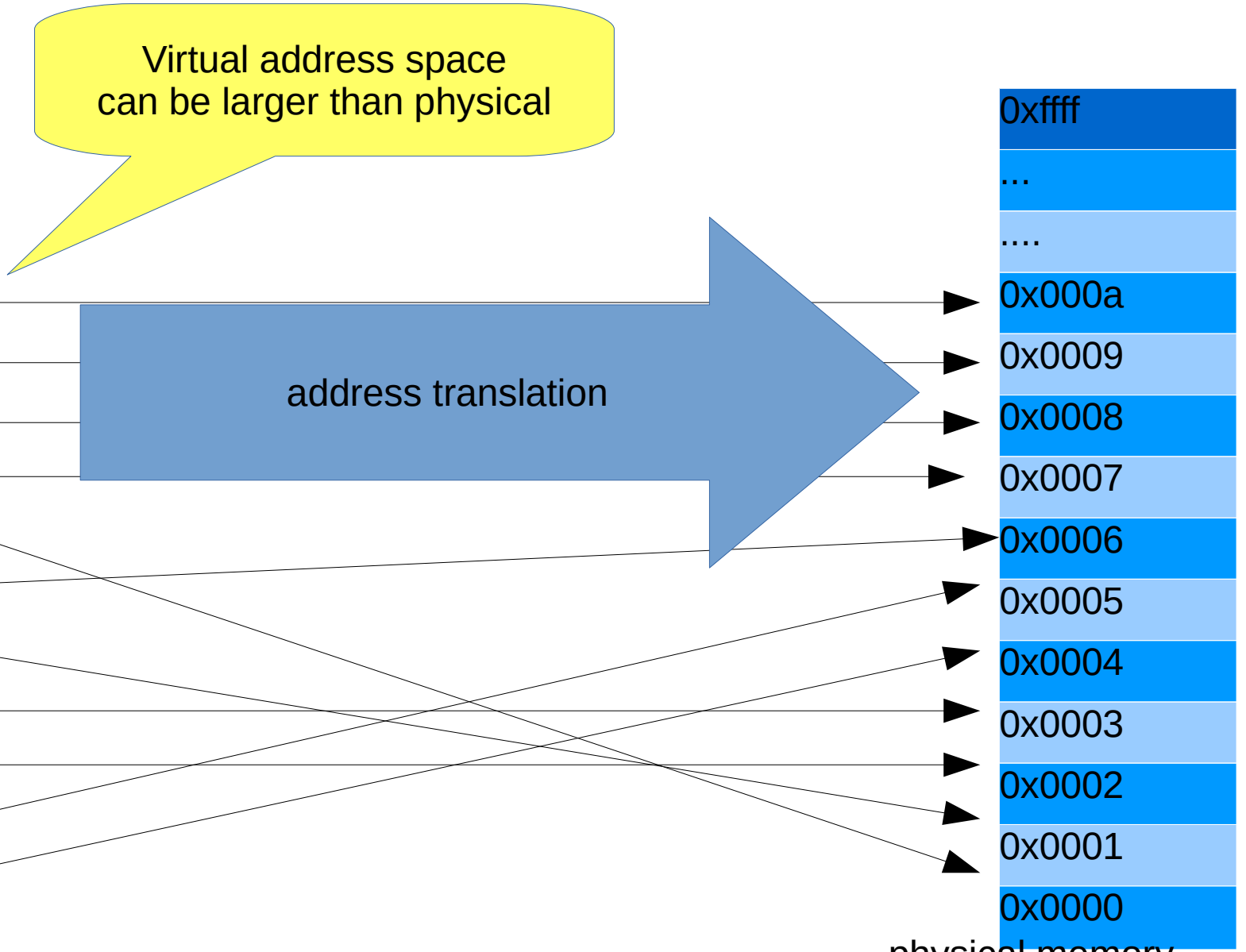
0xffffffff
...
....
0x0000000a
0x00000009
0x00000008
0x00000007
0x00000006
0x00000005
0x00000004
0x00000003
0x00000002
0x00000001
0x00000000

address translation

0xffff
...
....
0x000a
0x0009
0x0008
0x0007
0x0006
0x0005
0x0004
0x0003
0x0002
0x0001
0x0000

application address space

physical memory



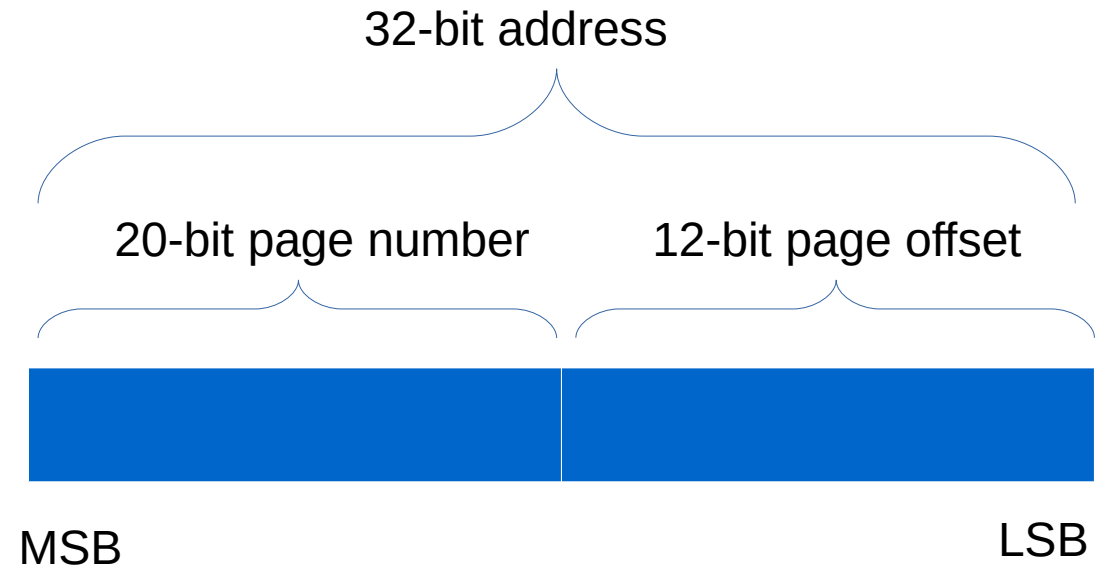
Virtual address	Physical address
Visible to applications	Not visible to applications; may be accessible to operating system
Used by <code>lw</code> and <code>sw</code> (or their equivalents)	
Used for fetching instructions from memory	

For every memory access, the virtual address must be translated to a physical address. This translation is done by special address translation hardware.

The translation happens automatically: the application program has no idea.

We divide address space (both virtual and physical) into *pages*. A typical size for pages is 4KB (4096, or 0x1000 bytes).

Address range	Page
0x0b000..0x0bfff	11
0x0a000..0x0afff	10
0x09000..0x09fff	9
0x08000..0x08fff	8
0x07000..0x07fff	7
0x06000..0x06fff	6
0x05000..0x05fff	5
0x04000..0x04fff	4
0x03000..0x03fff	3
0x02000..0x02fff	2
0x01000..0x01fff	1
0x00000..0x00fff	0

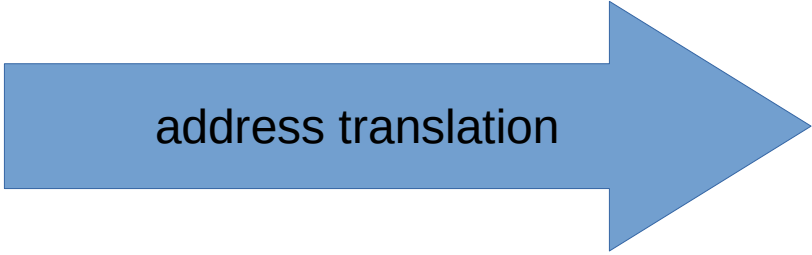


In other words:

$\text{page_num} = \text{addr} // \text{page_size}$

$\text{page_offset} = \text{addr} \% \text{page_size}$

virtual address

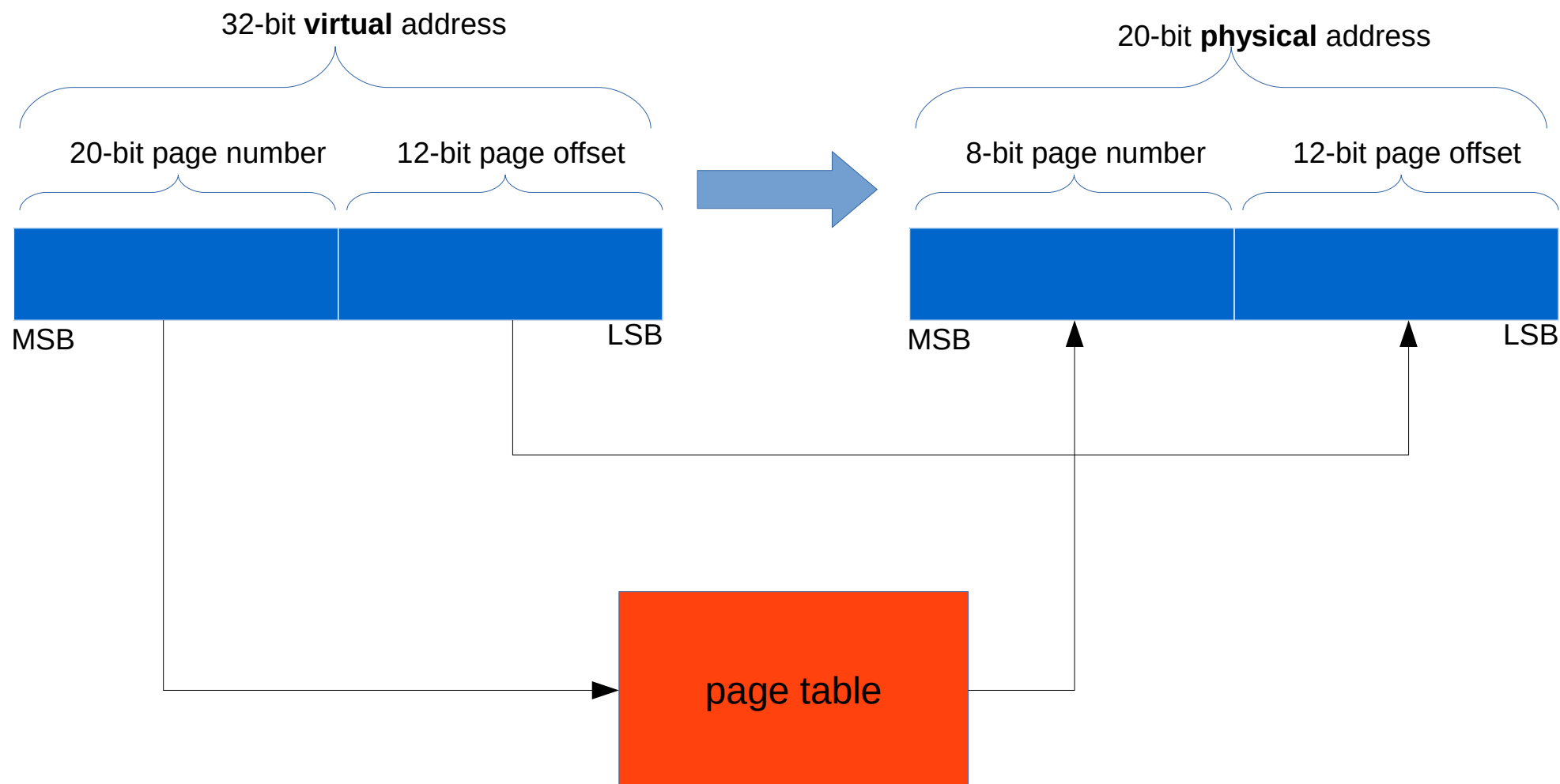


address translation

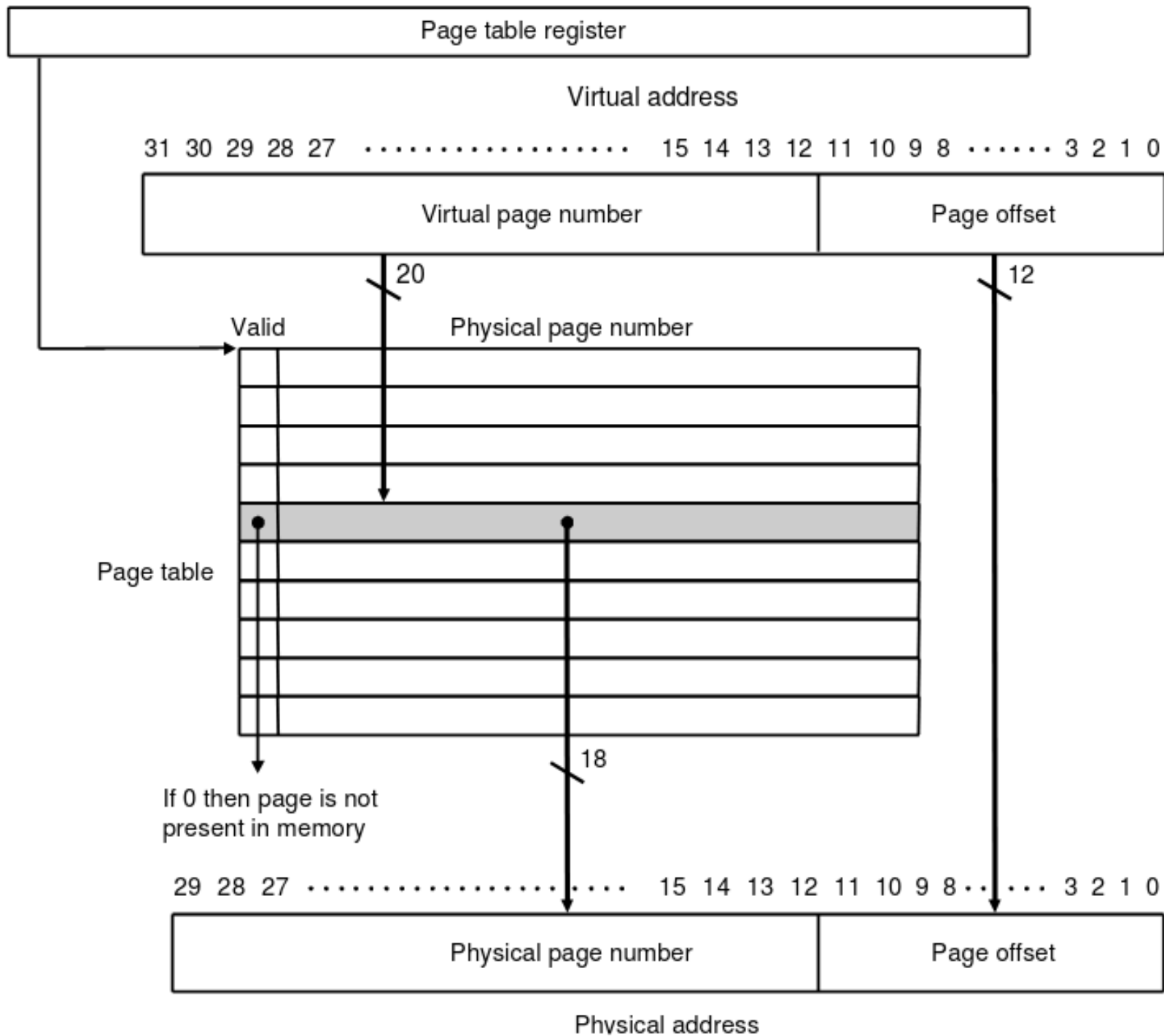
physical address

Address translation

When translating a virtual address to a physical address, we are really translating a *virtual page* to *physical page*. The offset stays the same.



The *page table* is a data structure mapping virtual page numbers to physical page numbers.



A page table

A page table is just an array. The index is the virtual page. Each virtual matches to a physical page.

Virtual Page	Physical Page
0	34
1	12
2	0
3	50
4	13
5	14
....	...

The page table is created and managed by the operating system.

Typically, each running program (process) will have its own page table, allowing each process to have its own address space.

There's usually a *special register* that holds the physical address of the page table. The CPU performs a page table lookup as part of its address translation on every memory access.

```
page_offset = virtual_addr % page_size
virtual_page = virtual_addr // page_size
physical_page = page_table[virtual_page]
physical_addr = physical_page * page_size + page_offset
```

A page table

Caveat: not all virtual pages have a corresponding physical page. If an application accesses such a page, the hardware signals to the OS that the memory access cannot proceed: this is a *page fault*.

Thus we had a "valid" bit, which is checked before each lookup.

Virtual Page	V	Physical Page
0	1	34
1	0	
2	1	0
3	1	50
4	1	13
5	1	14
....		...

```
page_offset = virtual_addr % page_size
```

```
virtual_page = virtual_addr // page_size
```

```
physical_page = page_table[virtual_page]["physical_page"] if page_table[virtual_page]["v"]
```

```
physical_addr = physical_page * page_size + page_offset
```


Compiler

Source code

```
int x;  
void foo()  
{  
    x = 3;  
}
```

Machine code

```
sw 3, 0x814020
```

OS

Page table

VirtualPage	V	PhysicalPage
...		
0x813	0	
0x814	1	0x99
0x815	0	
....		

On application load,
initialize page table.
Select physpage
based on availability.

CPU

PhysAddr

0x99020

VirtAddr

0x814020

On execution, virtual
address is translated
via page table.

We assume here 4096-byte pages.

What is the total size (in bits) of a page table for this system?

Assume a **32-bit virtual address**, consisting of a 20-bit virtual page number and a 12-bit offset.

Assume a **20-bit physical address**, consisting of an 8-bit physical page number and a 12-bit offset.

Assume the page table contains a physical page and valid bit for each entry.

What is the total size (in bits) of a page table for this system?

Assume a **32-bit virtual address**, consisting of a 20-bit virtual page number and a 12-bit offset.

Assume a **20-bit physical address**, consisting of an 8-bit physical page number and a 12-bit offset.

Assume the page table contains a physical page and valid bit for each entry.

How many entries are in the page table?

$$2^{**}20 = 1048576$$

What is the total size (in bits) of a page table for this system?

Assume a **32-bit virtual address**, consisting of a 20-bit virtual page number and a 12-bit offset.

Assume a **20-bit physical address**, consisting of an 8-bit physical page number and a 12-bit offset.

Assume the page table contains a physical page and valid bit for each entry.

How many entries are in the page table?

$$2^{**}20 = 1048576$$

How many bits is each entry?

1 valid bit + 8-bit physical page number

What is the total size (in bits) of a page table for this system?

Assume a **32-bit virtual address**, consisting of a 20-bit virtual page number and a 12-bit offset.

Assume a **20-bit physical address**, consisting of an 8-bit physical page number and a 12-bit offset.

Assume the page table contains a physical page and valid bit for each entry.

How many entries are in the page table?

$$2^{20} = 1048576$$

How many bits is each entry?

1 valid bit + 8-bit physical page number

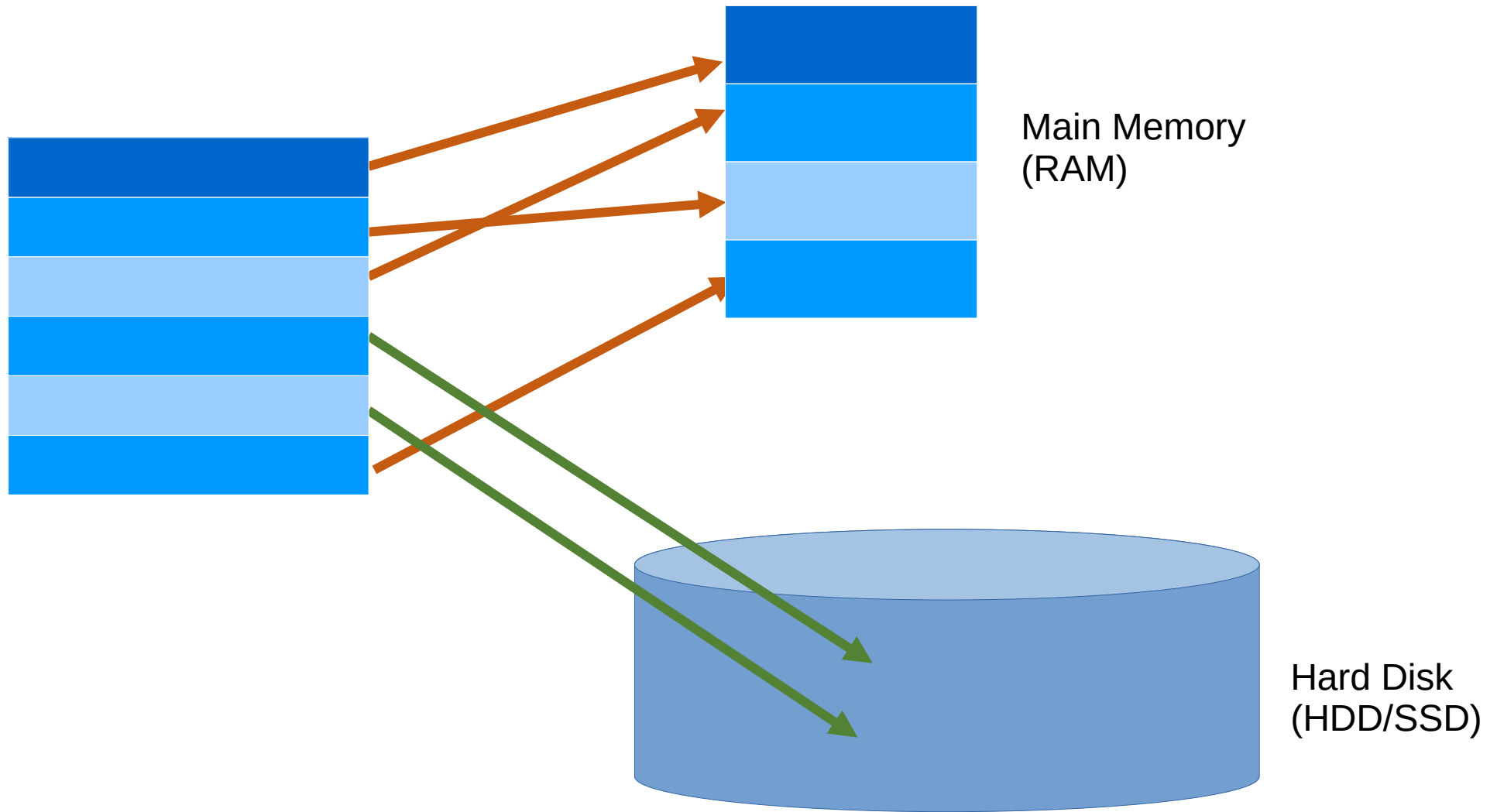
What is the total size of the page table?

$$2^{20} * (1 + 8) = 9437184 \text{ bits}$$

Index	V(1-bit)	Physical Page (8-bits)
0		
1		
2		
3		
.		
.		
.		
.		
.		
2 ²⁰ - 1		

Note that we don't need to store Virtual Page Number or index because we can calculate it using $VPN = VA // \text{page_size}$

Virtual Memory

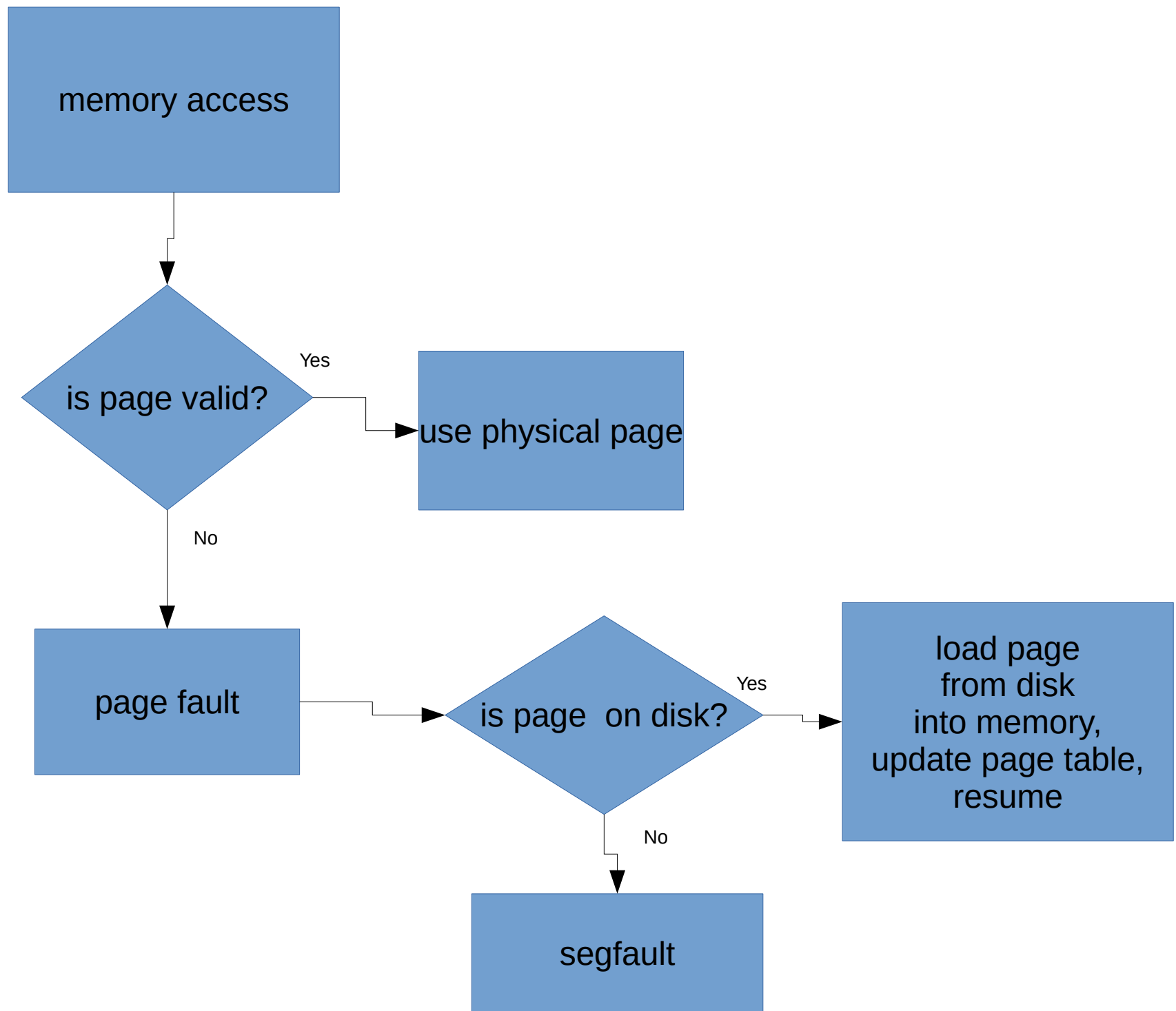


Anything does not fit on Main Memory is stored in Hard Disk

The swapping mechanism hopefully will:
give us the **size** of the disk,
with the **speed** of memory.

The design of virtual memory is based on the slowness of disk access.

We want to keep reduce misses.



Swapping

Swapping

The **OS** is responsible for allocating memory to applications.

If there is **memory pressure** (i.e. programs are using all available physical memory), what can the OS do?

Reject memory requests. Can't start programs, existing processes fail.

Or,

- The OS can choose to evict pages from memory and *swap* them to disk.
- The corresponding page will be marked as invalid, so that when it's accessed, it will generate a page fault, whereupon it will be swapped back into memory.
- The OS should prefer to swap seldom-used pages.

A page table

When we evict a page from memory, we don't always have to save it to disk:

- The page may be empty or unused.
- The page may have been loaded from a file (e.g. an executable), so there's no reason to save it to disk again.
- The page may already be on disk, from when it was previously swapped out.

In none of the above cases apply, we have to write the page to disk.

Virtual Page	V	Dirty	Physical Page
0	1	1	34
1	0		
2	1	0	0
3	1	1	50
4	1	0	13
5	1	0	14
....			...

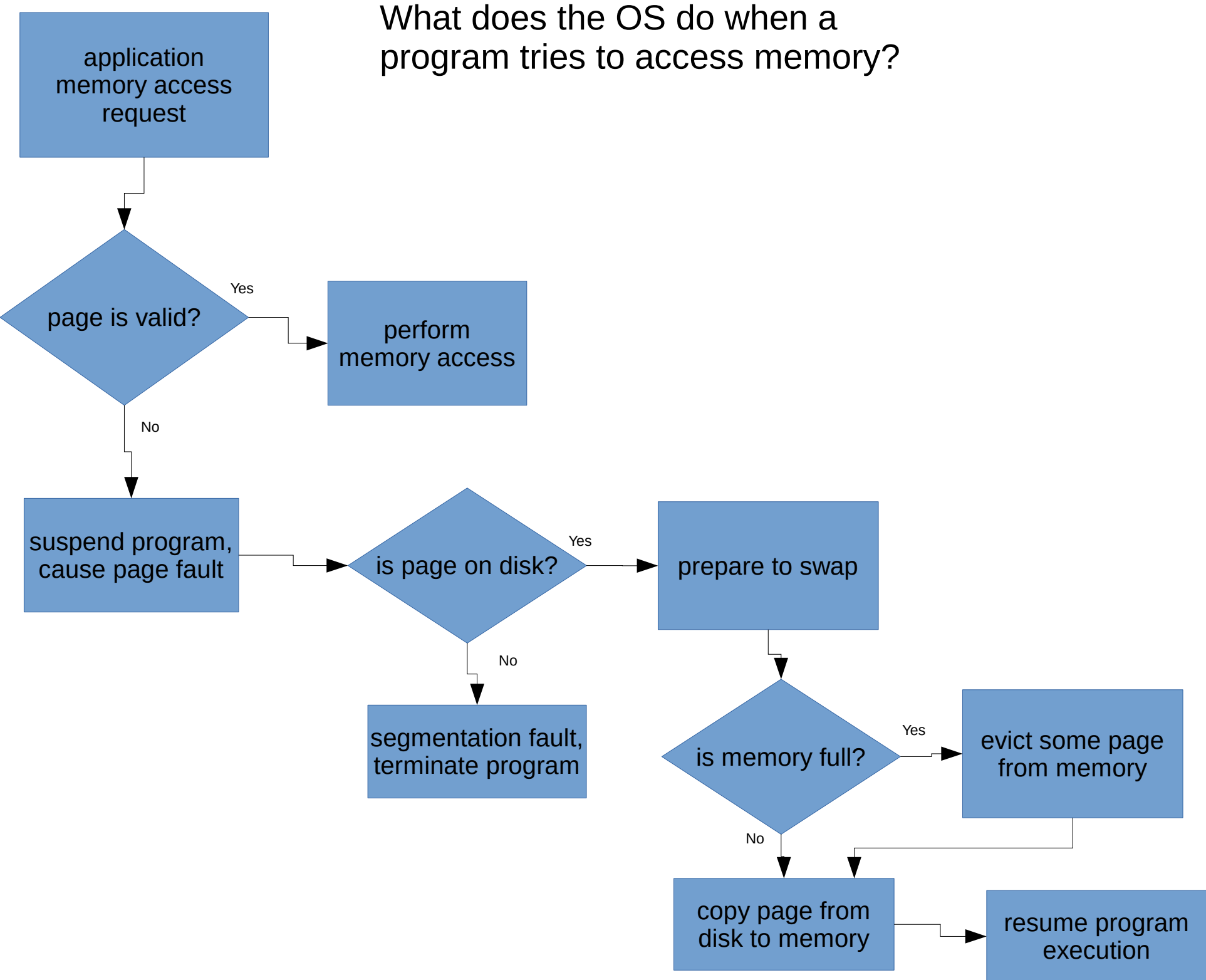
Every page initially has its dirty bit set to zero.

When a program does a write to that page, the processor sets the dirty bit to 1.

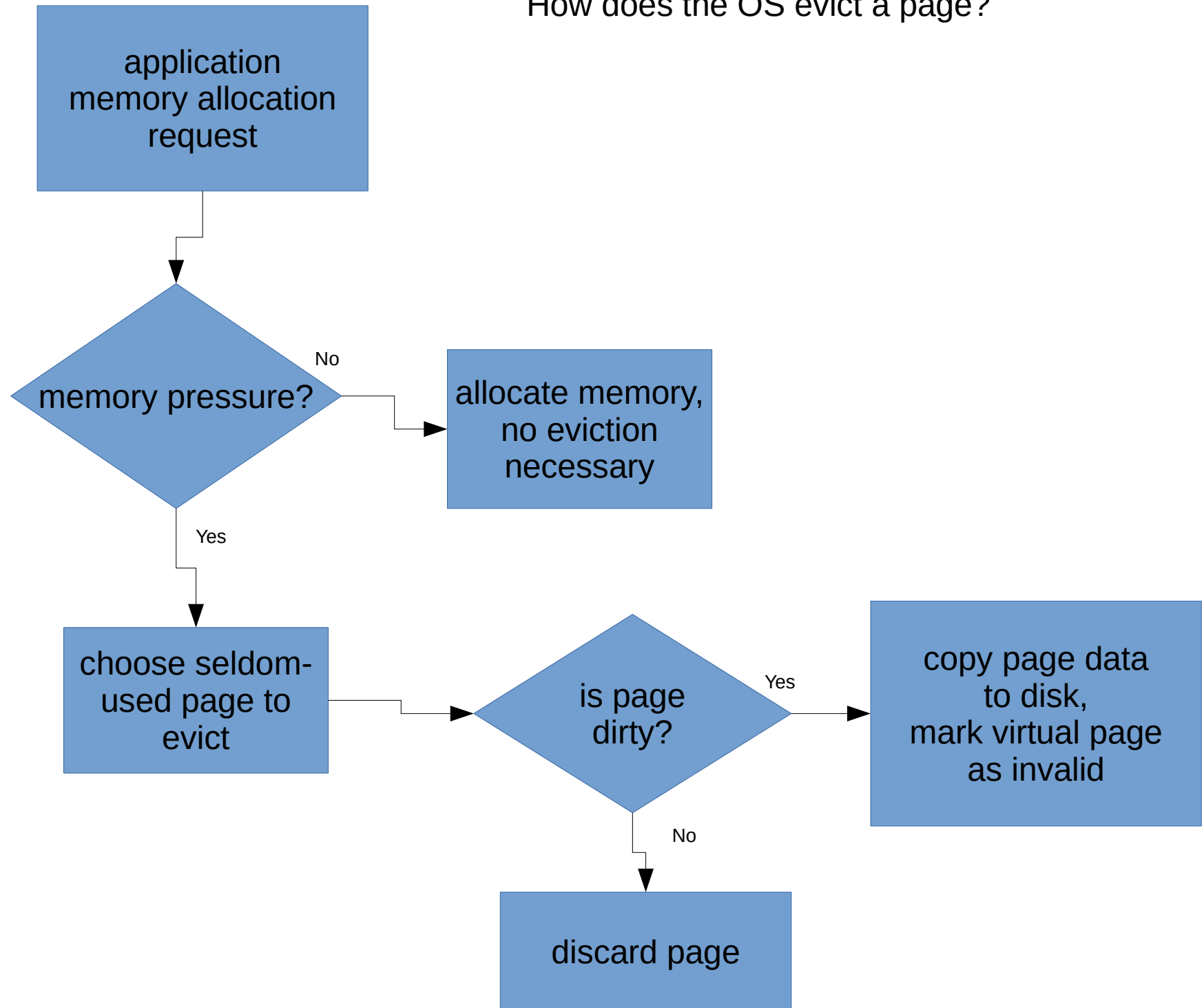
The OS will write the page to disk when swapping only if the dirty bit is 1.

This should all sound very familiar! With paging, *memory is a write-back cache of disk.*

What does the OS do when a program tries to access memory?



How does the OS evict a page?



OS and applications may share address space:
here, both address spaces overlap at physical
page 0.

Reasons for this: the OS exposes code to the
application for *system calls*.

What can go wrong?

0xffff
...
...
0x0004
0x0003
0x0002
0x0001
0x0000

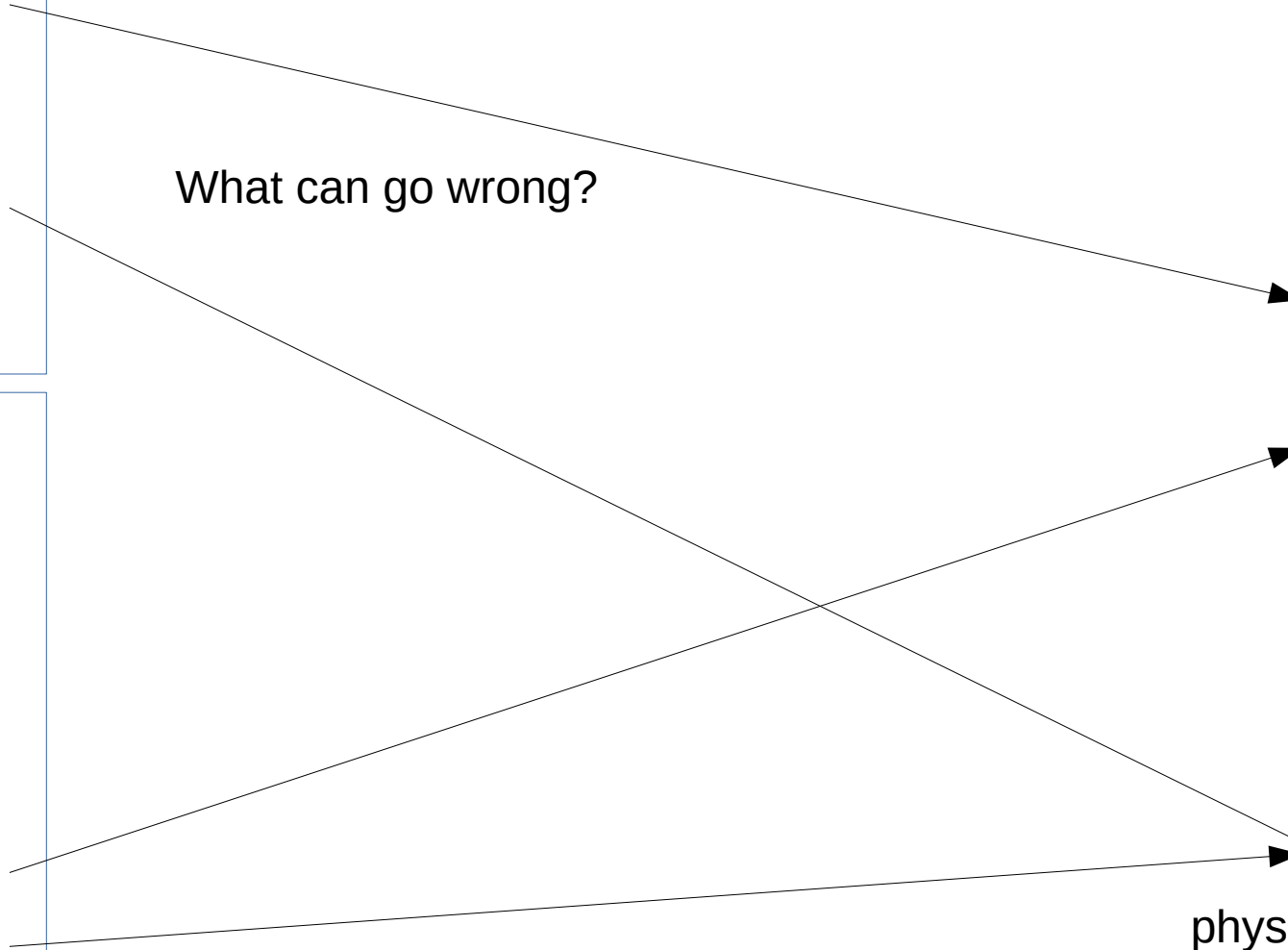
application
address space

0xffff
...
...
0x0004
0x0003
0x0002
0x0001
0x0000

OS address space

0xffff
...
...
0x000a
0x0009
0x0008
0x0007
0x0006
0x0005
0x0004
0x0003
0x0002
0x0001
0x0000

physical memory



A page table

To prevent the application from changing OS data, we introduce a *protect bit*. Pages marked with it cannot be written.

Virtual Page	V	Dirty	Protect	Physical Page
0	1	1	1	34
1	0			
2	1	0	0	0
3	1	1	0	50
4	1	0	0	13
5	1	0	0	14
....				...

A page table

Assume an 8-byte page size. For each of the following address, given a corresponding physical page, or indicate if a page fault will occur. If the page is not writeable, indicate so.

43, 32, 10, 5, 30

Virtual Page	V	Dirty	Protect	Physical Page
0	1	1	1	34
1	0			
2	1	0	0	0
3	1	1	0	50
4	1	0	0	13
5	1	0	0	14
....				...

Page Offset = Virtual Address % Page_Size

Virtual Page Number (VPN) = VA // Page_Size

Physical Page Number (PPN) = PageTable[VPN] if V == 1.

Physical Address = PPN * Page_Size + Offset

A page table

Assume an 8-byte page size. For each of the following address, given a corresponding physical page, or indicate if a page fault will occur. If the page is not writeable, indicate so.

43, 32, 10, 5, 30

Virtual Page	V	Dirty	Protect	Physical Page
0	1	1	1	34
1	0			
2	1	0	0	0
3	1	1	0	50
4	1	0	0	13
5	1	0	0	14
....				...

$\text{Offset} = \text{VA} \% \text{Page_Size}$
 $\text{VPN} = \text{VA} // \text{Page_Size}$
 $\text{PPN} = \text{PageTable}[\text{VPN}]$
if $V == 1$.
 $\text{PA} = \text{PPN} * \text{Page_Size} + \text{Offset}$

$\text{Offset} = 43 \% 8 = 3$
 $\text{VPN} = 43 // 8 = 5$
 $\text{PPN} = \text{PageTable}[5]$
 $= 14$
 $\text{PA} = 14 * 8 + 3$
 $= 115$

1. virtual address 43. Page 5, offset 3. Physical address 115.

A page table

Assume an 8-byte page size. For each of the following address, given a corresponding physical page, or indicate if a page fault will occur. If the page is not writeable, indicate so.

43, 32, 10, 5, 30

Virtual Page	V	Dirty	Protect	Physical Page
0	1	1	1	34
1	0			
2	1	0	0	0
3	1	1	0	50
4	1	0	0	13
5	1	0	0	14
....				...

$\text{Offset} = \text{VA} \% \text{Page_Size}$
 $\text{VPN} = \text{VA} // \text{Page_Size}$
 $\text{PPN} = \text{PageTable}[\text{VPN}]$
 if $V == 1$.
 $\text{PA} = \text{PPN} * \text{Page_Size} + \text{Offset}$

$\text{Offset} = 32 \% 8 = 0$
 $\text{VPN} = 32 // 8 = 4$
 $\text{PPN} = \text{PageTable}[4]$
 $= 13$
 $\text{PA} = 13 * 8 + 0$
 $= 104$

1. virtual address 43. Page 5, offset 3. Physical address 115.
2. virtual address 32. Page 4, offset 0. Physical address 104.

A page table

Assume an 8-byte page size. For each of the following address, given a corresponding physical page, or indicate if a page fault will occur. If the page is not writeable, indicate so.

43, 32, 10, 5, 30

Virtual Page	V	Dirty	Protect	Physical Page
0	1	1	1	34
1	0			
2	1	0	0	0
3	1	1	0	50
4	1	0	0	13
5	1	0	0	14
....				...

$\text{Offset} = \text{VA} \% \text{Page_Size}$
 $\text{VPN} = \text{VA} // \text{Page_Size}$
 $\text{PPN} = \text{PageTable}[\text{VPN}]$
if $V == 1$.
 $\text{PA} = \text{PPN} * \text{Page_Size} + \text{Offset}$

$\text{Offset} = 10 \% 8 = 2$
 $\text{VPN} = 10 // 8 = 1$
Valid bit is 0,
Page fault

1. virtual address 43. Page 5, offset 3. Physical address 115.
2. virtual address 32. Page 4, offset 0. Physical address 104.
3. virtual address 10. Page 1, offset 2. Page fault!

A page table

Assume an 8-byte page size. For each of the following address, given a corresponding physical page, or indicate if a page fault will occur. If the page is not writeable, indicate so.

43, 32, 10, 5, 30

Virtual Page	V	Dirty	Protect	Physical Page
0	1	1	1	34
1	0			
2	1	0	0	0
3	1	1	0	50
4	1	0	0	13
5	1	0	0	14
....				...

$\text{Offset} = \text{VA} \% \text{Page_Size}$
 $\text{VPN} = \text{VA} // \text{Page_Size}$
 $\text{PPN} = \text{PageTable}[\text{VPN}]$
 if $V == 1$.
 $\text{PA} = \text{PPN} * \text{Page_Size} + \text{Offset}$

$\text{Offset} = 5 \% 8 = 5$
 $\text{VPN} = 5 // 8 = 0$
 $\text{PPN} = \text{PageTable}[0]$
 $= 34$
 $\text{PA} = 34 * 8 + 5$
 $= 277$
 Protect-bit is 1,
 so read only.

1. virtual address 43. Page 5, offset 3. Physical address 115.
2. virtual address 32. Page 4, offset 0. Physical address 104.
3. virtual address 10. Page 1, offset 2. Page fault!
4. virtual address 5. Page 0, offset 5. Physical address 277. Read only.
5. virtual address 30. Page 3, offset 6. Physical address 406.

A page table

Assume an 8-byte page size. For each of the following address, given a corresponding physical page, or indicate if a page fault will occur. If the page is not writeable, indicate so.

43, 32, 10, 5, 30

Virtual Page	V	Dirty	Protect	Physical Page
0	1	1	1	34
1	0			
2	1	0	0	0
3	1	1	0	50
4	1	0	0	13
5	1	0	0	14
....				...

$\text{Offset} = \text{VA} \% \text{Page_Size}$
 $\text{VPN} = \text{VA} // \text{Page_Size}$
 $\text{PPN} = \text{PageTable}[\text{VPN}]$
 if $V == 1$.
 $\text{PA} = \text{PPN} * \text{Page_Size} + \text{Offset}$

$\text{Offset} = 30 \% 8 = 6$
 $\text{VPN} = 30 // 8 = 3$
 $\text{PPN} = \text{PageTable}[3]$
 $= 50$
 $\text{PA} = 50 * 8 + 6$
 $= 406$

1. virtual address 43. Page 5, offset 3. Physical address 115.
2. virtual address 32. Page 4, offset 0. Physical address 104.
3. virtual address 10. Page 1, offset 2. Page fault!
4. virtual address 5. Page 0, offset 5. Physical address 277. Read only.
5. virtual address 30. Page 3, offset 6. Physical address 406.

Program walkthrough

- 1) When you run a program from an executable file, the content of the file is copied into memory. In this case, the program machine occupies virtual pages 0 and 1. They are valid, because they are in memory, and they are not dirty, because their content is directly copied from the file.

Virtual Page Num	Valid?	Dirty?
0	1	0
1	1	0
2	0	0
3	0	0
4	0	0
5	0	0
6	0	0

On disk:
program.exe -> {0,1}

Program walkthrough

- 1) When you run a program from an executable file, the content of the file is copied into memory. In this case, the program machine occupies virtual pages 0 and 1. They are valid, because they are in memory, and they are not dirty, because their content is directly copied from the file.
- 2) As the program runs, it requests additional memory from the operating system. The OS handles this by mapping additional pages 2 and 3 into its address space. They are now marked as valid. They are empty, and so are not dirty.

Virtual Page Num	Valid?	Dirty?
0	1	0
1	1	0
2	1	0
3	1	0
4	0	0
5	0	0
6	0	0

On disk:
program.exe -> {0,1}

Program walkthrough

- 1) When you run a program from an executable file, the content of the file is copied into memory. In this case, the program machine occupies virtual pages 0 and 1. They are valid, because they are in memory, and they are not dirty, because their content is directly copied from the file.
- 2) As the program runs, it requests additional memory from the operating system. The OS handles this by mapping additional pages 2 and 3 into its address space. They are now marked as valid. They are empty, and so are not dirty.
- 3) When the program writes to these new pages, they are marked dirty.

Virtual Page Num	Valid?	Dirty?
0	1	0
1	1	0
2	1	1
3	1	1
4	0	0
5	0	0
6	0	0

On disk:
program.exe -> {0,1}

Program walkthrough

- 1) When you run a program from an executable file, the content of the file is copied into memory. In this case, the program machine occupies virtual pages 0 and 1. They are valid, because they are in memory, and they are not dirty, because their content is directly copied from the file.
- 2) As the program runs, it requests additional memory from the operating system. The OS handles this by mapping additional pages 2 and 3 into its address space. They are now marked as valid. They are empty, and so are not dirty.
- 3) When the program writes to these new pages, they are marked dirty.
- 4) Now, some other application causes memory pressure, and the OS decides to swap out our program's pages 2 and 3 to disk. They are now marked invalid and not dirty.

Virtual Page Num	Valid?	Dirty?
0	1	0
1	1	0
2	0	0
3	0	0
4	0	0
5	0	0
6	0	0

On disk:
program.exe -> {0,1}
swapfile -> {2,3}

Program walkthrough

- 1) When you run a program from an executable file, the content of the file is copied into memory. In this case, the program machine occupies virtual pages 0 and 1. They are valid, because they are in memory, and they are not dirty, because their content is directly copied from the file.
- 2) As the program runs, it requests additional memory from the operating system. The OS handles this by mapping additional pages 2 and 3 into its address space. They are now marked as valid. They are empty, and so are not dirty.
- 3) When the program writes to these new pages, they are marked dirty.
- 4) Now, some other application causes memory pressure, and the OS decides to swap out our program's pages 2 and 3 to disk. They are now marked invalid and not dirty.

Why would the OS swap out 2 and 3, instead of (non-dirty) 0 and 1?

It's possible the program was still running, resulting in reads to machine code in pages 0 and 1. Therefore pages 0 and 1 may be used more recently than 2 and 3.

Virtual Page Num	Valid?	Dirty?
0	1	0
1	1	0
2	0	0
3	0	0
4	0	0
5	0	0
6	0	0

On disk:
program.exe -> {0,1}
swapfile -> {2,3}

Program walkthrough

- 1) When you run a program from an executable file, the content of the file is copied into memory. In this case, the program machine occupies virtual pages 0 and 1. They are valid, because they are in memory, and they are not dirty, because their content is directly copied from the file.
- 2) As the program runs, it requests additional memory from the operating system. The OS handles this by mapping additional pages 2 and 3 into its address space. They are now marked as valid. They are empty, and so are not dirty.
- 3) When the program writes to these new pages, they are marked dirty.
- 4) Now, some other application causes memory pressure, and the OS decides to swap out our program's pages 2 and 3 to disk. They are now marked invalid and not dirty.
- 5) When our program tries to access those pages, it will cause a page fault. The OS will swap them back in and mark them valid. They are still marked not dirty, because the copy on disk is still present.

Virtual Page Num	Valid?	Dirty?
0	1	0
1	1	0
2	1	0
3	1	0
4	0	0
5	0	0
6	0	0

On disk:
program.exe -> {0,1}
swapfile -> {2,3}

Program walkthrough

- 1) When you run a program from an executable file, the content of the file is copied into memory. In this case, the program machine occupies virtual pages 0 and 1. They are valid, because they are in memory, and they are not dirty, because their content is directly copied from the file.
- 2) As the program runs, it requests additional memory from the operating system. The OS handles this by mapping additional pages 2 and 3 into its address space. They are now marked as valid. They are empty, and so are not dirty.
- 3) When the program writes to these new pages, they are marked dirty.
- 4) Now, some other application causes memory pressure, and the OS decides to swap out our program's pages 2 and 3 to disk. They are now marked invalid and not dirty.
- 5) When our program tries to access those pages, it will cause a page fault. The OS will swap them back in and mark them valid. They are still marked not dirty, because the copy on disk is still present.
- 6) If the OS decides again detects memory pressure, it is safe to simply discard these pages, so copies already exist on disk.

Virtual Page Num	Valid?	Dirty?
0	1	0
1	1	0
2	0	0
3	0	0
4	0	0
5	0	0
6	0	0

On disk:
program.exe -> {0,1}
swapfile -> {2,3}

Summary of today's lecture

- Today we cover the following topics.
 - What is virtual memory? Why we use virtual memory?
 - Virtual Address
 - Virtual Address to Physical Address Mapping/Translation
 - Page Table
 - Calculate the Size of Page Table (Polls)
 - Page Fault
 - Segmentation Fault
 - How to handle Memory Pressure?
 - How to Protect OS Data?
 - Examples of Calculating Physical Address from Virtual's Address.
 - Program Walk Through
- Reference: Lecture Slide on Brightspace. File Name: 08-vm.pdf
- Next Class:
 - Multilevel Page Lookup
 - TLB
 - Intel x86 Architecture and Register Model

How big is a page table?

Consider a system with 32-bit virtual addresses, 32-bit physical addresses, and 4KB pages. How big will the page tables be (including metadata)? Assume each memory cell is one byte.

How big is a page table?

Consider a system with 32-bit virtual addresses, 32-bit physical addresses, and 4KB pages. How big will the page tables be (including metadata)? Assume each memory cell is one byte.

There will be 2^{20} page table entries.

Each entry will contain the 20-bit physical page, plus the dirty, protect, and valid bits.

So the total size is $2^{20} * (20 + 3) = 24,117,248$ bits, or 3,014,656 bytes.

Each process will have a page table. That adds up!

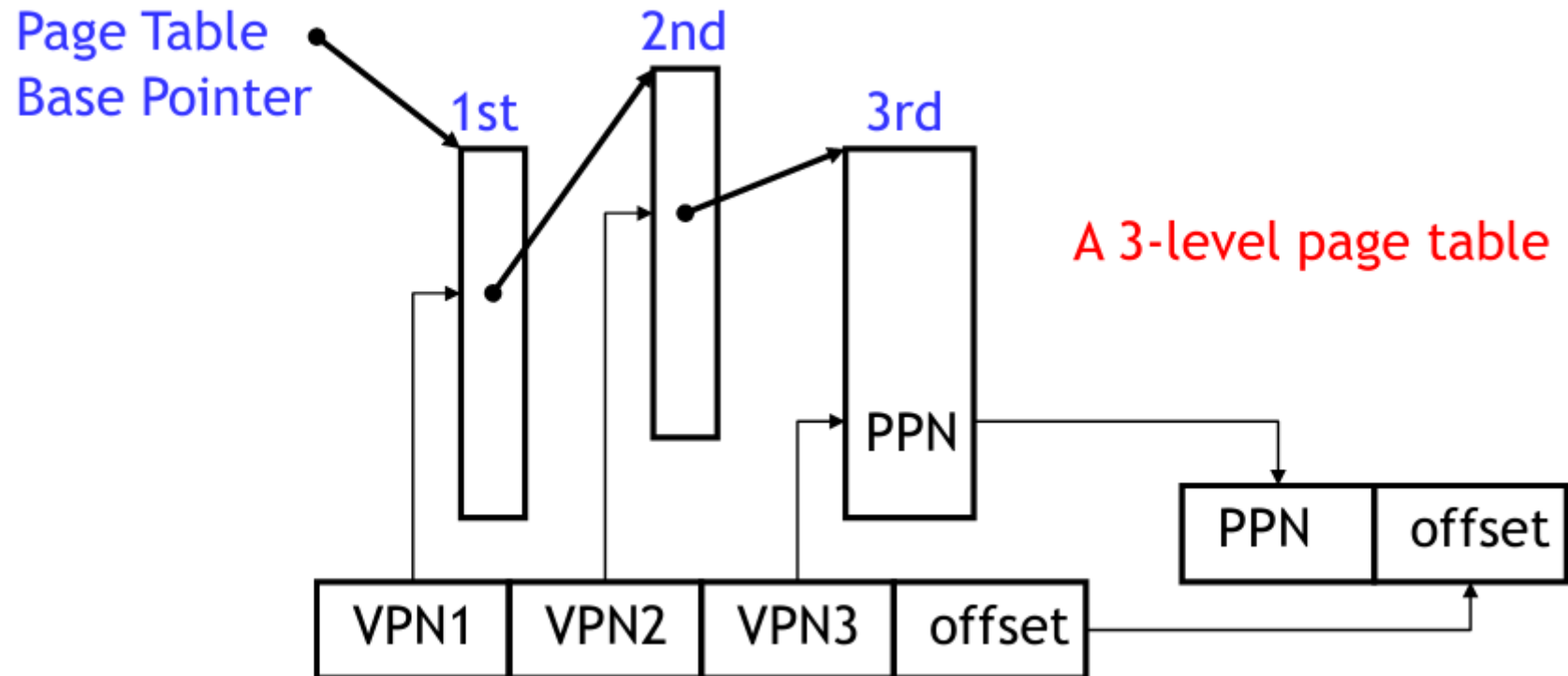
Further observation: most programs use only a small part of their address space. Most page tables are mostly empty.

Another observation: with a page table, every memory access by an application entails *two* (2) round-trip accesses to memory: first to translate the address in the page table, then to access the actual address. So we've just cut the speed of our memory unit in half.

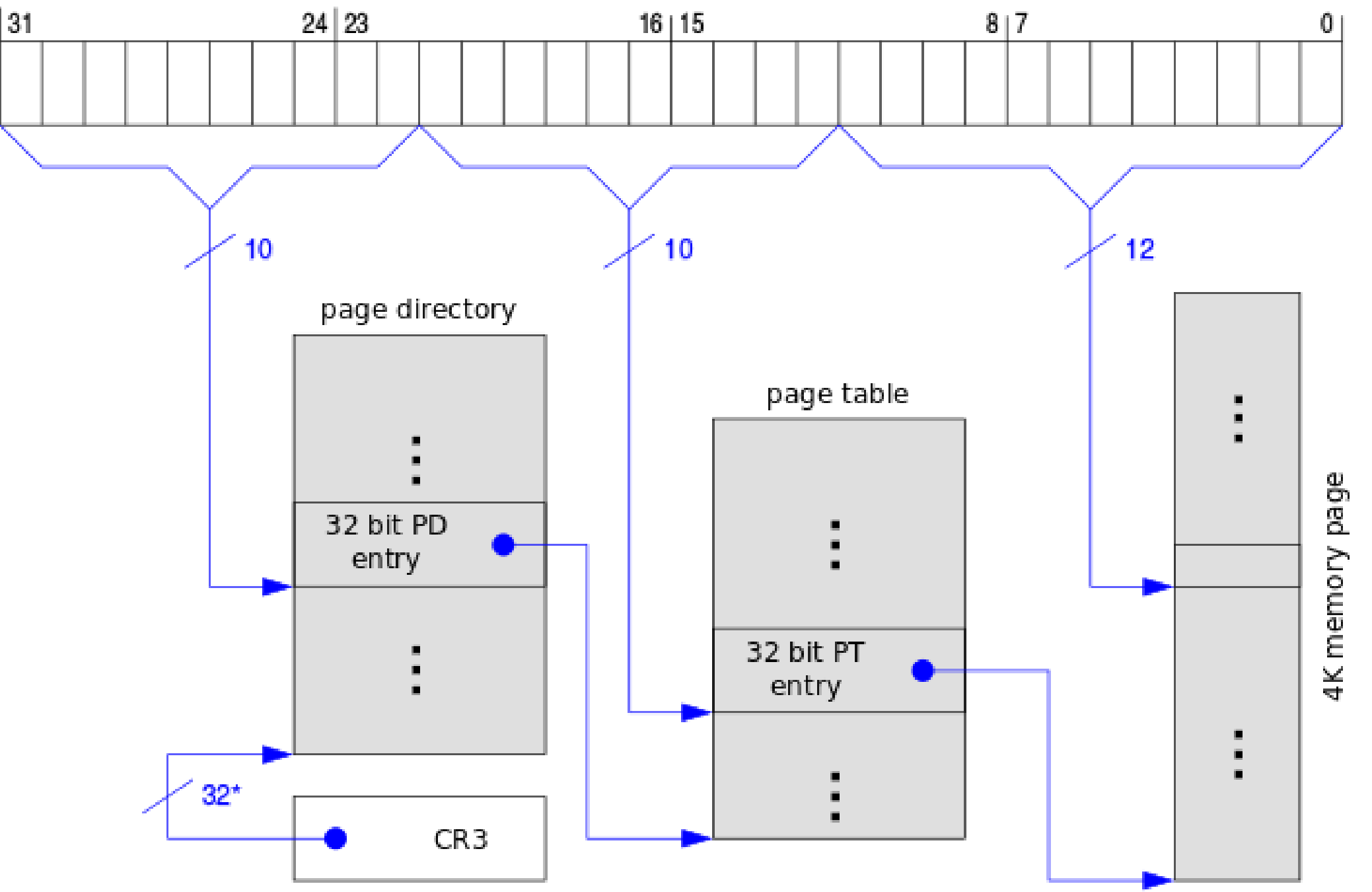
Page tables

- Solutions to too-big page table
 - We swap it out, of course
 - The location of the page table is stored by another page table
 - We thus have a hierarchy of page tables, much as we had a hierarchy of caches
- Solutions to too-slow page table
 - We cache it, of course
 - Virtual-to-physical page translations are cached in the Translation Lookaside Buffer (TLB)

Multilevel Page Lookup



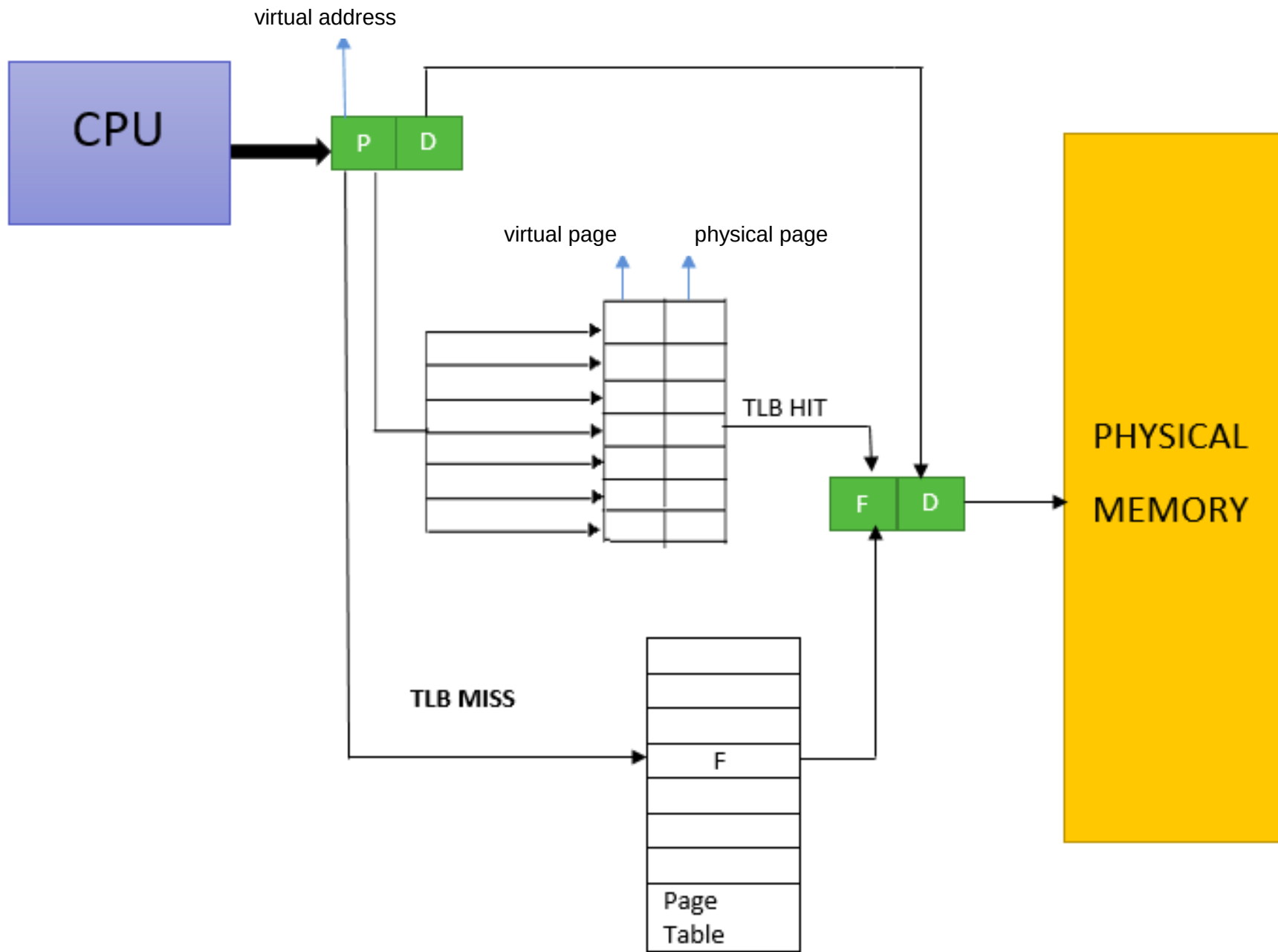
Linear address:



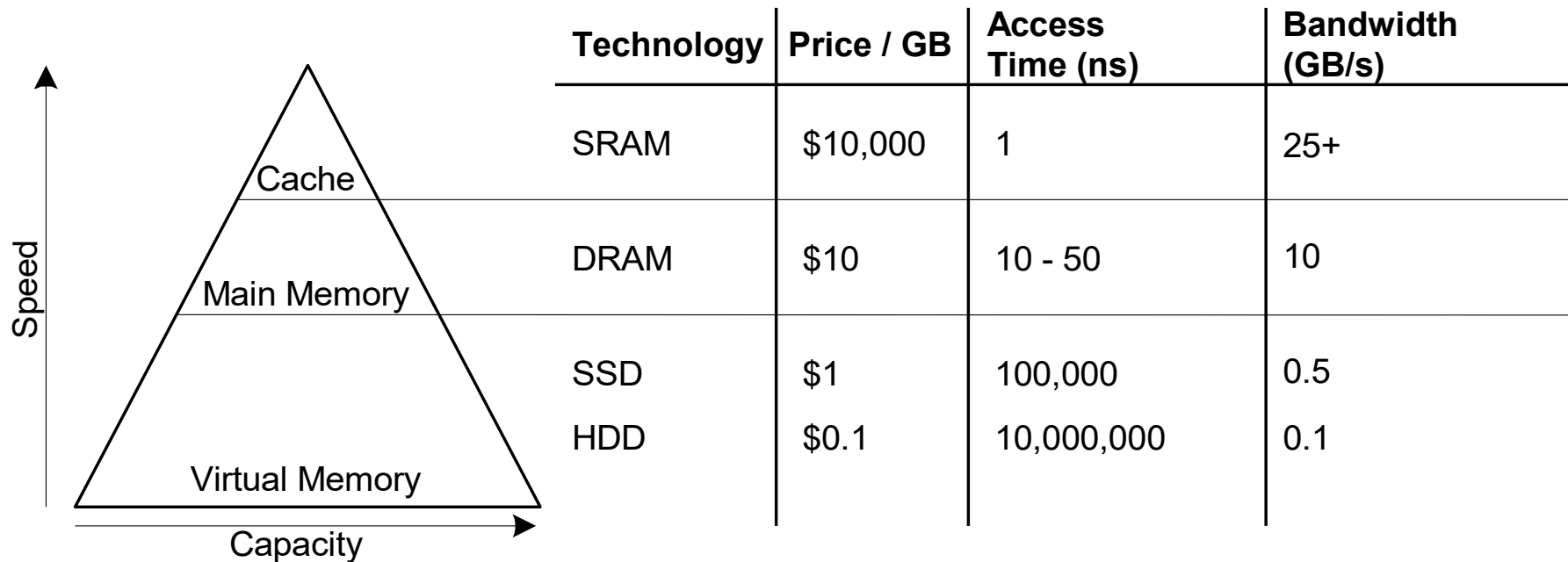
*) 32 bits aligned to a 4-KByte boundary

TLB

- The TLB (Translation Lookaside Buffer) is a cache of the page table
- It maps virtual addresses to physical addresses
 - The tag is the virtual page
 - The value stored is the physical page
 - Also has valid, dirty, and protect bits
 - TLB is consulted before checking the page table
- It's usually a fully-associative cache.



Memory Hierarchy



- **System:**

- Virtual memory size: 4 GB = 2^{32} bytes
- Physical memory size: 1 GB = 2^{30} bytes
- Page size: 4 KB = 2^{12} bytes

- **Organization:**

- Virtual address: **32** bits
- Physical address: **30** bits
- Page offset: **12** bits
- # Virtual pages = $2^{32}/2^{12} = 2^{20}$ (VPN = 20 bits)
- # Physical pages = $2^{30}/2^{12} = 2^{18}$ (PPN = 18 bits)

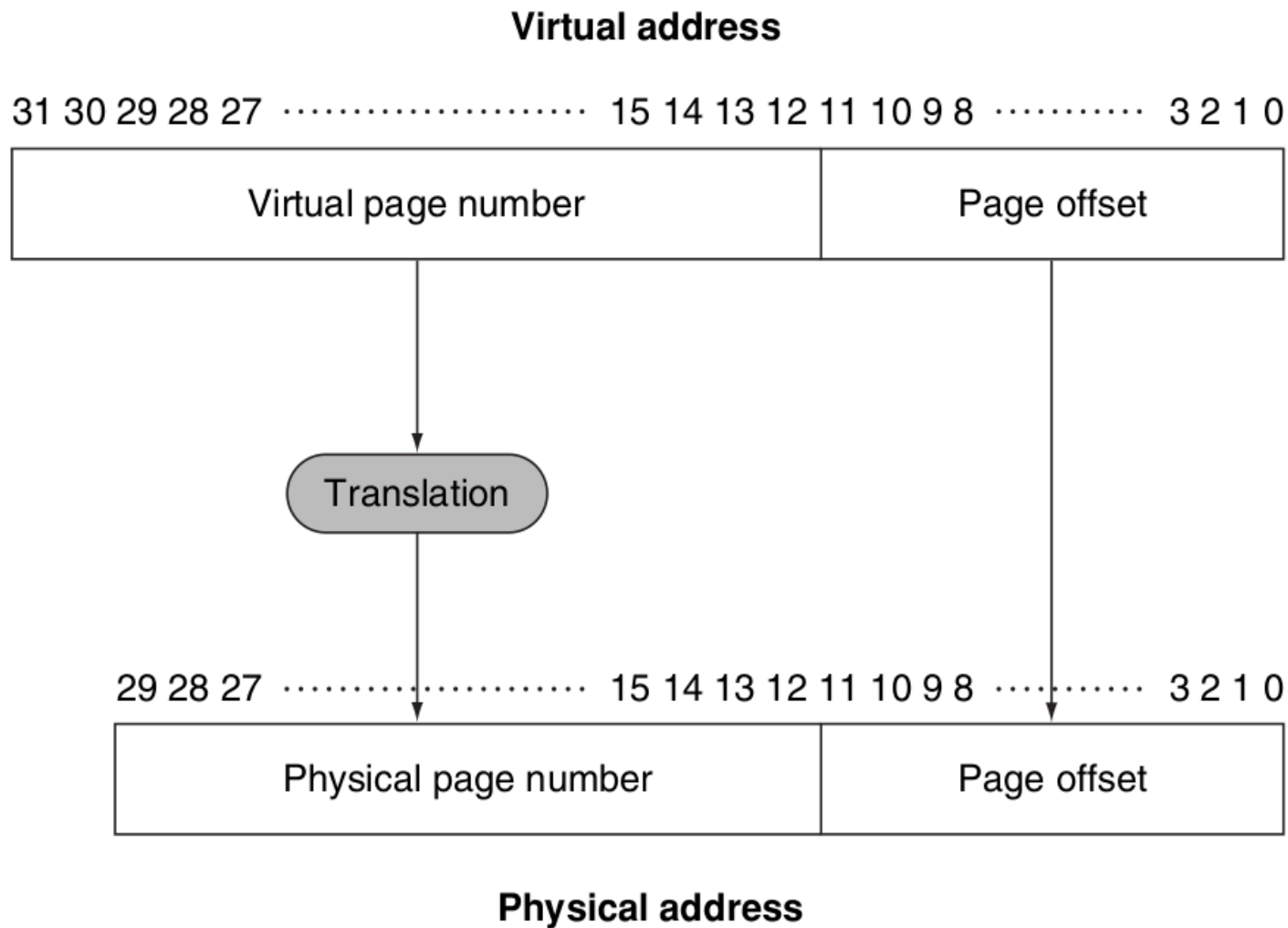
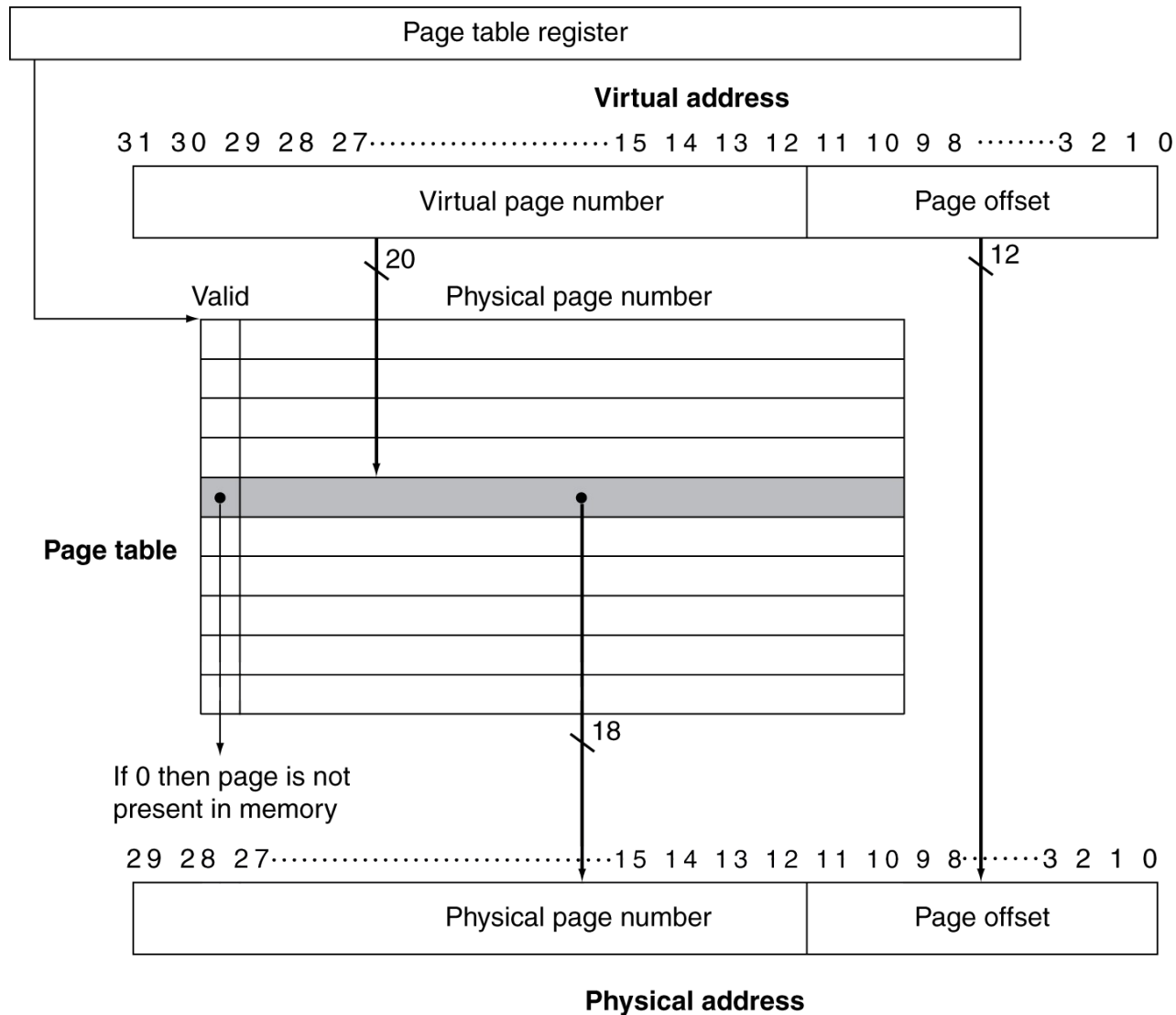
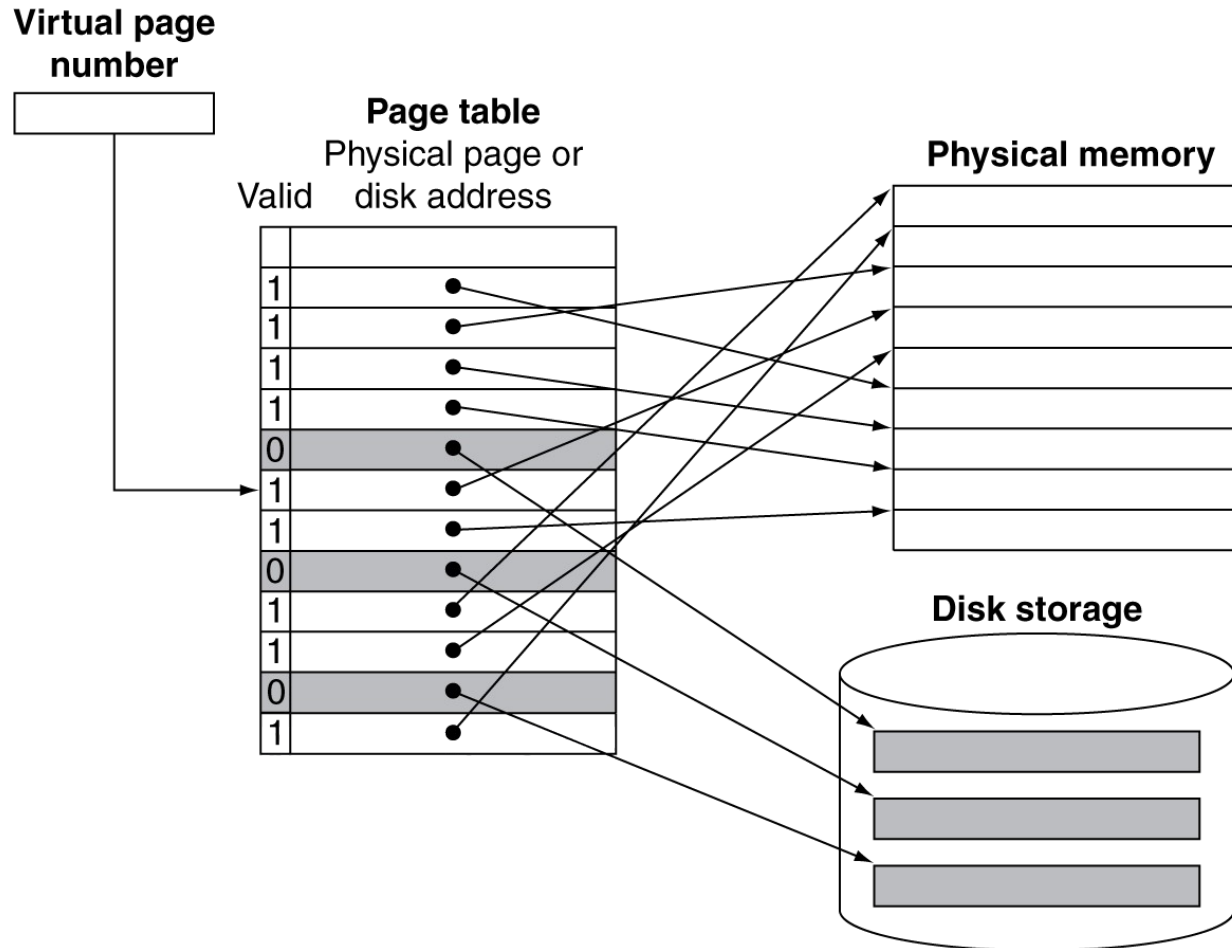


FIGURE 5.26 Mapping from a virtual to a physical address. The page size is $2^{12} = 4$ KiB. The number of physical pages allowed in memory is 2^{18} , since the physical page number has 18 bits in it. Thus, main memory can have at most 1 GiB, while the virtual address space is 4 GiB.

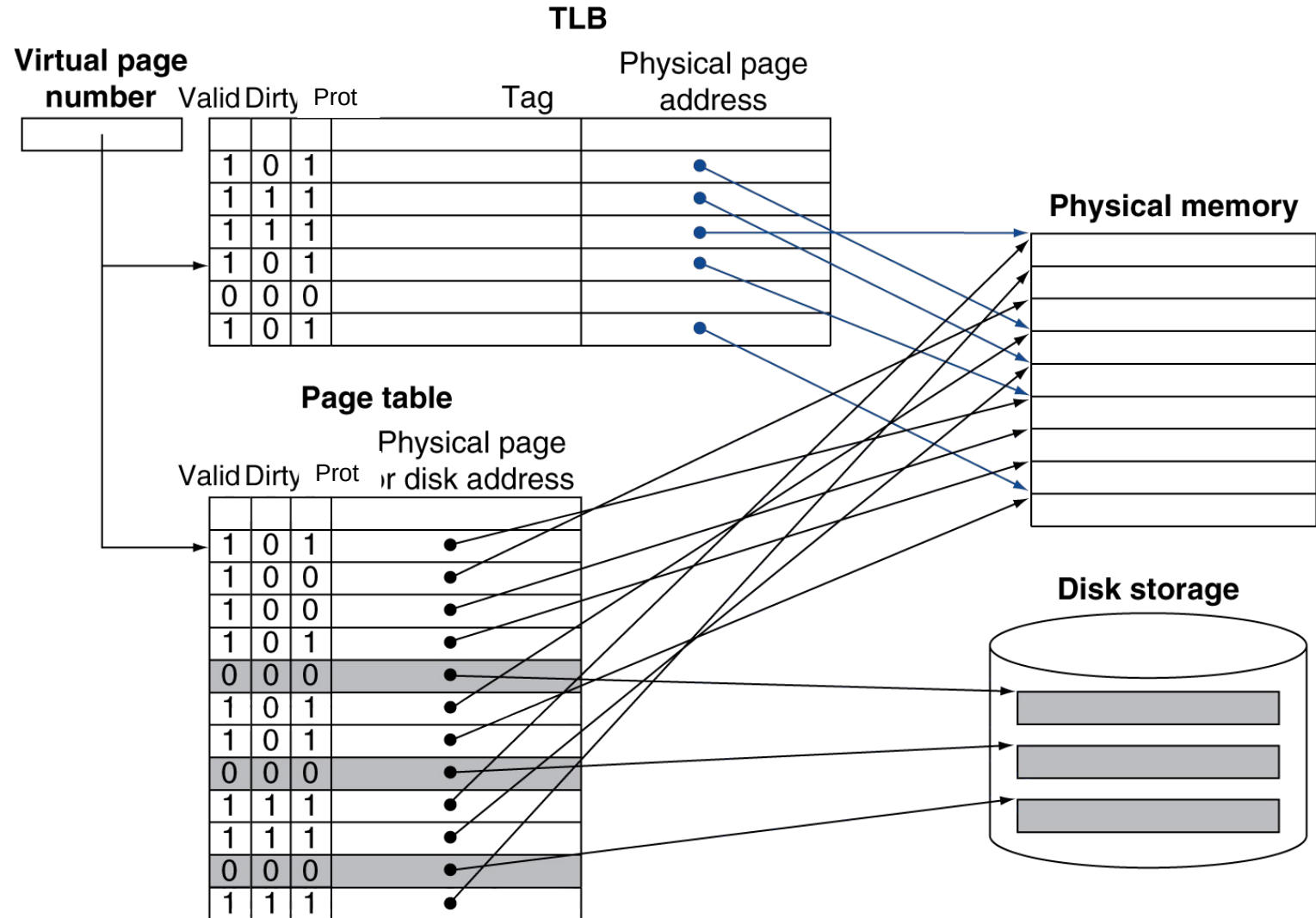
Translation Using a Page Table



Mapping Pages to Storage



Fast Translation Using a TLB



Hierarchical page tables

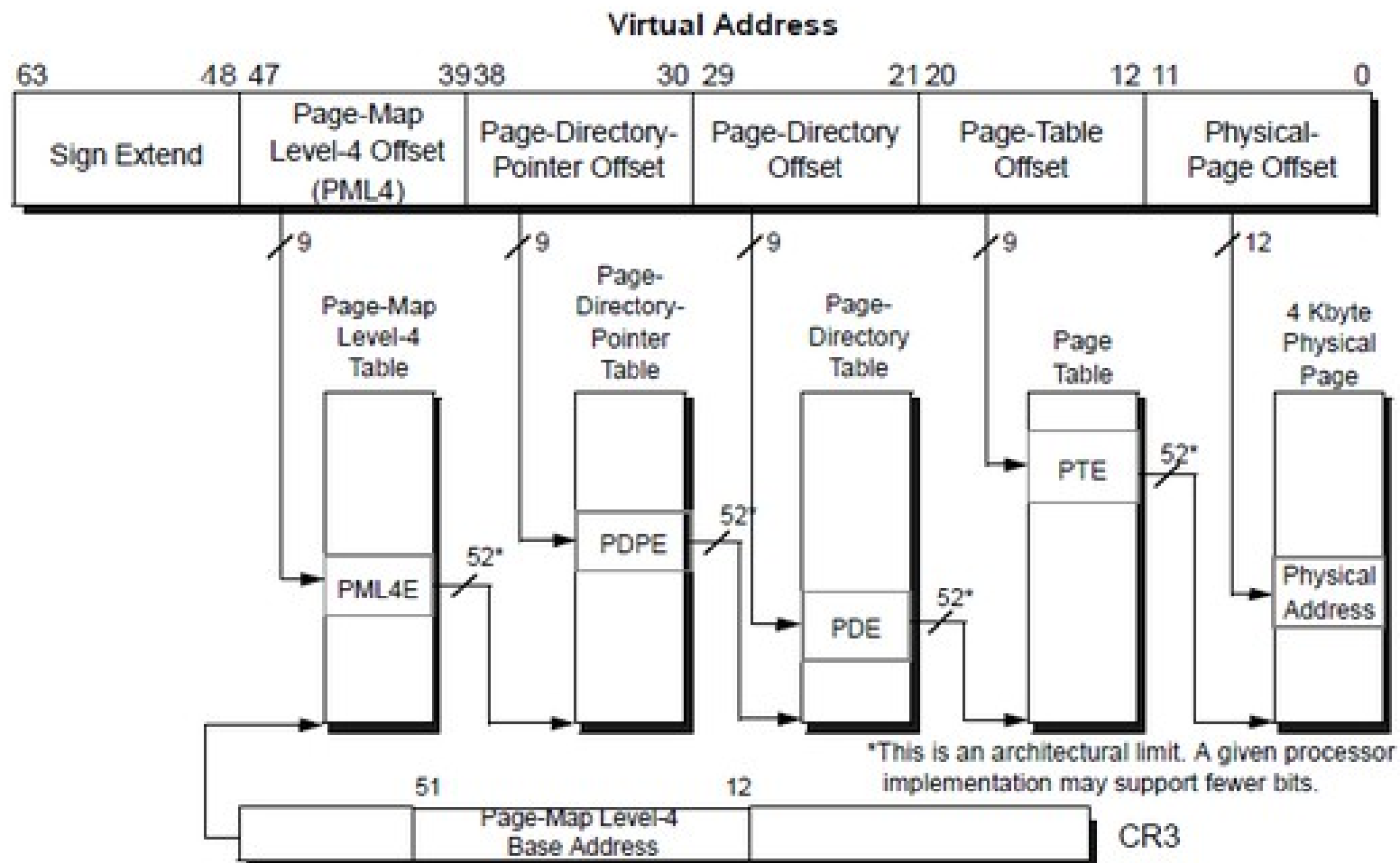
Modern 64-bit Intel-based systems have 4 or 5 levels of page tables.

Each layer has a 9-bit index.

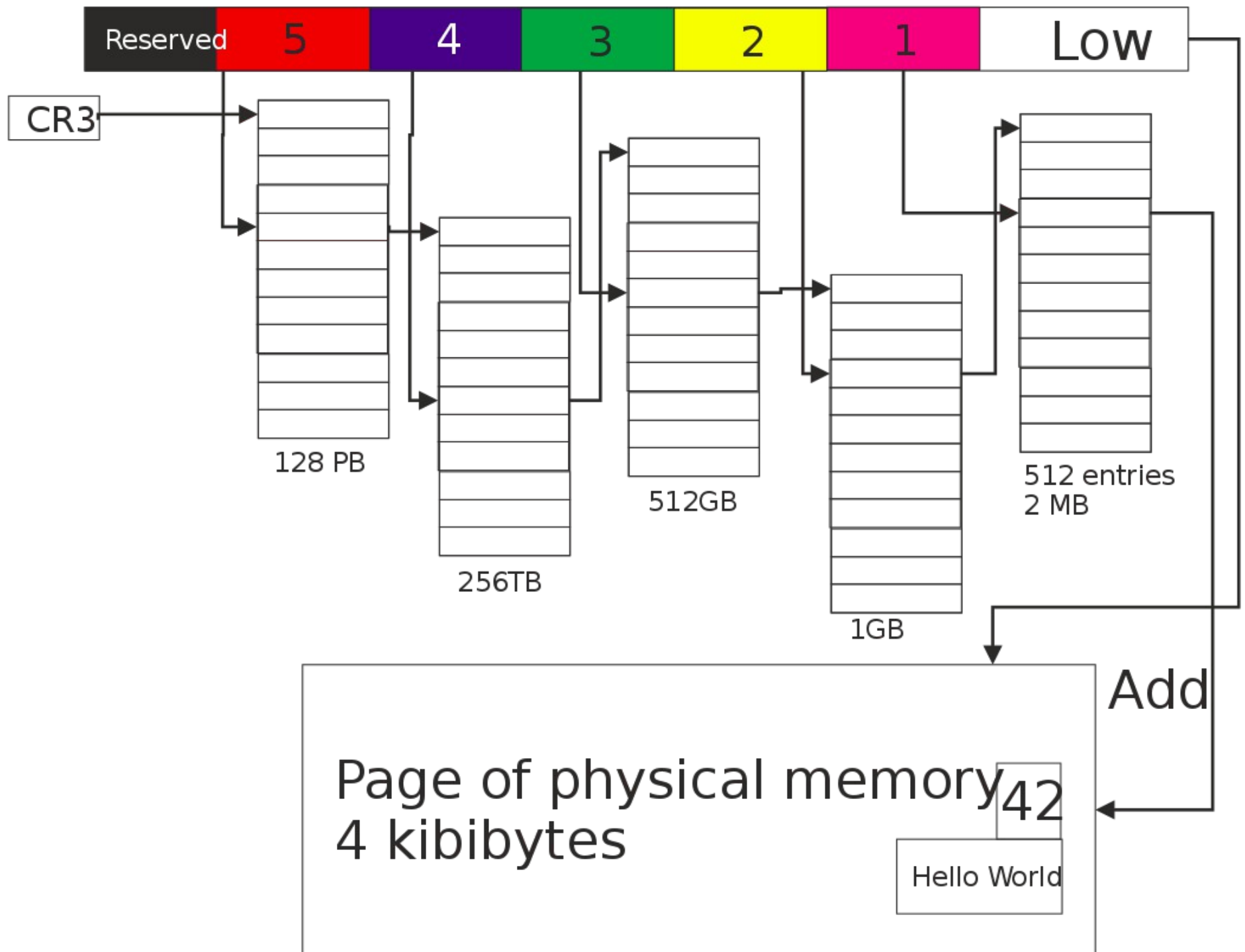
4-level page tables allow virtual address of 48 bits, address space of 256 TiB.

5-level page tables allow virtual address of 57 bits, address space of 128 PiB.

Typically, low virtual address are for userland, high virtual addresses are reserved for kernel.



Virtual Address



Note that the most significant bits of virtual addresses are unused. Rather than simply be zero, they are *sign extended*, that is, they must all have the same value as the most significant used bit of the virtual address.